N045 AI 기능을 서비스로 설계하기_해설서

Chapter01

강의정리 챕터 01. AI 기능을 서비스로 설계하기

1. 강의 목적

AI 기능을 한 번 만들어 보는 것과 사람들이 지속적으로 사용하는 서비스로 만드는 것은 서로 다른 문제입니다.

코딩 에이전트를 이용하면 AI 기능을 빠르게 구현할 수 있지만, 실제 서비스로 제공하려면 사용자가 접근하는 방법, 데이터 보안, API 비용, AI 오류 대응, 시스템 구조까지 함께 설계해야 합니다.

이 강의의 핵심은 AI 기능을 만드는 관점에서 벗어나, 사용자가 지속적으로 이용할 수 있는 AI 서비스를 설계하는 관점으로 전환하는 것입니다.

- 오전: 사용 맥락 분석, 제공 채널 선택, 처리 방식 및 오류 대응, 코드 없는 프로토타입 테스트
- 오후: 아키텍처 다이어그램, 시퀀스 다이어그램, 데이터 모델 이해 및 설계
- 최종 결과물: 서비스의 핵심 의사결정을 담은 한 장짜리 PRD(제품 요구사항 문서)

※ 이번 챕터는 배포나 설치보다 서비스 설계와 문서화에 집중합니다.

2. 중학생도 이해할 수 있는 설명

2-1. AI 기능과 AI 서비스는 다릅니다.

식당을 예로 들어 보겠습니다.

- AI 기능 만들기: 맛있는 음식 한 가지를 개발합니다.
- AI 서비스 만들기: 손님이 음식을 주문하고, 주방에서 조리하고, 결제하고, 문제가 생기면 해결하는 전체 과정을 설계합니다.

음식이 맛있어도 주문하기 어렵거나, 가격이 지나치게 비싸거나, 문제가 생겼을 때 해결할 방법이 없다면 손님은 다시 오지 않습니다.

AI도 마찬가지입니다. 답변을 잘 생성하는 기능만으로는 지속 가능한 서비스가 되기 어렵습니다.

2-2. 서비스 설계의 다섯 가지 질문

① 누가, 언제, 왜 사용하는가?

사용 맥락을 정의합니다. 혼자 쓰는지, 직원들이 함께 쓰는지, 고객에게 공개하는지에 따라 설계가 달라집니다.

② 어떤 경로로 제공할 것인가?

기성 AI 도구, 업무용 도구, 자동화 워크플로, 간단한 인터페이스, 독립적인 웹앱 등 적합한 제공 채널을 선택합니다.

③ 어떤 방식으로 처리할 것인가?

즉시 결과를 보여 줄지, 백그라운드에서 처리할지, 정해진 시간에 자동 실행할지 결정합니다.

④ 문제가 발생하면 어떻게 할 것인가?

AI의 잘못된 답변, API 오류, 시간 초과, 과도한 비용, 민감정보 노출에 대비합니다.

⑤ 내부에서 어떤 구성 요소가 어떻게 연결되는가?

화면, 서버, AI 모델, 데이터베이스가 어떤 순서로 정보를 주고받는지 설계합니다.

이 다섯 가지 질문에 답하면 단순한 AI 실험을 실제 서비스로 발전시키기 위한 기본 틀이 만들어집니다.

3. 핵심 개념 정리

3-1. 제공 채널 선택

서비스의 목적과 사용 환경에 따라 적절한 제공 방법을 선택해야 합니다.

제공 방식	적합한 상황	장단점
기성 AI 도구	개인이 빠르게 실험할 때	개발이 거의 필요 없지만 맞춤화에 한계
업무용 AI 도구	조직 내부에서 업무를 지원할 때	기존 업무와 연계하기 좋지만 권한·보안 검토 필요
자동화 워크플로	정해진 여러 작업을 연결할 때	반복 업무에 효율적이나 예외 처리 설계 필요
간단한 사용자 인터페이스	특정 업무를 쉽게 수행하게 할 때	사용하기 쉽지만 기능 확장에 제약이 있을 수 있음

제공 방식	적합한 상황	장단점
독립형 웹앱	여러 사용자에게 독립적인 서비스를 제공할 때	자유도가 높지만 개발·운영·보안 부담이 큼

핵심 원칙은 처음부터 웹앱을 개발하지 말고, 가장 단순한 방법으로 사용 가치를 검증한 뒤 필요한 만큼 복잡도를 높이는 것입니다.

3-2. 처리 방식 선택

동기 처리(Synchronous)

사용자가 요청하면 처리가 끝날 때까지 기다렸다가 결과를 받습니다.

예: 짧은 글 요약, 간단한 번역, 짧은 질의응답

비동기 처리(Asynchronous)

작업을 맡긴 뒤 다른 일을 할 수 있고, 완료되면 결과를 확인합니다.

예: 긴 문서 분석, 여러 파일 처리, 대량 콘텐츠 생성

스케줄 실행(Scheduled)

미리 정한 시간이나 주기에 따라 자동으로 실행합니다.

예: 매일 아침 뉴스레터 생성, 주간 시장 동향 분석

처리 방식은 단순히 AI가 빠른지 느린지로 결정하지 않습니다. 작업 시간, 사용자의 대기 가능 시간, 실패 시 복구 방법, 실행 비용을 함께 고려해야 합니다.

3-3. AI 오류 대응

AI는 항상 정확한 결과를 보장하지 않습니다. 따라서 정상 작동뿐 아니라 실패했을 때의 행동까지 설계해야 합니다.

- 재시도: 일시적인 네트워크 오류나 API 오류가 발생하면 제한된 횟수만큼 다시 실행합니다.
- 시간 제한: 무한정 기다리지 않도록 최대 처리 시간을 설정합니다.
- 대체 경로(Fallback): AI 호출에 실패하면 기본 기능이나 다른 모델로 전환합니다.
- 검증: 결과가 비어 있거나 형식이 잘못되었거나 근거가 부족하면 사용자에게 알립니다.
- 비용 제한: 사용자별 호출 한도, 사용량 추적, 예산 경보를 설정합니다.
- 사람의 확인: 중요한 외부 전송이나 되돌리기 어려운 작업은 실행 전에 승인을 받습니다.

특히 회사 문서를 외부 AI에 전송하는 서비스라면 데이터의 민감도, 외부 전송 여부, 보관 정책, 접근 권한을 먼저 확인해야 합니다.

3-4. 코드 없는 사용성 테스트

서비스를 개발하기 전에 사용자가 실제로 필요로 하는지 검증할 수 있습니다.

오즈의 마법사(Wizard of Oz) 테스트는 사용자에게는 서비스가 작동하는 것처럼 보여 주지만, 내부에서는 사람이 일부 작업을 대신 수행하는 방식입니다.

예를 들어 AI 스토리 코치 서비스를 만든다고 가정하겠습니다.

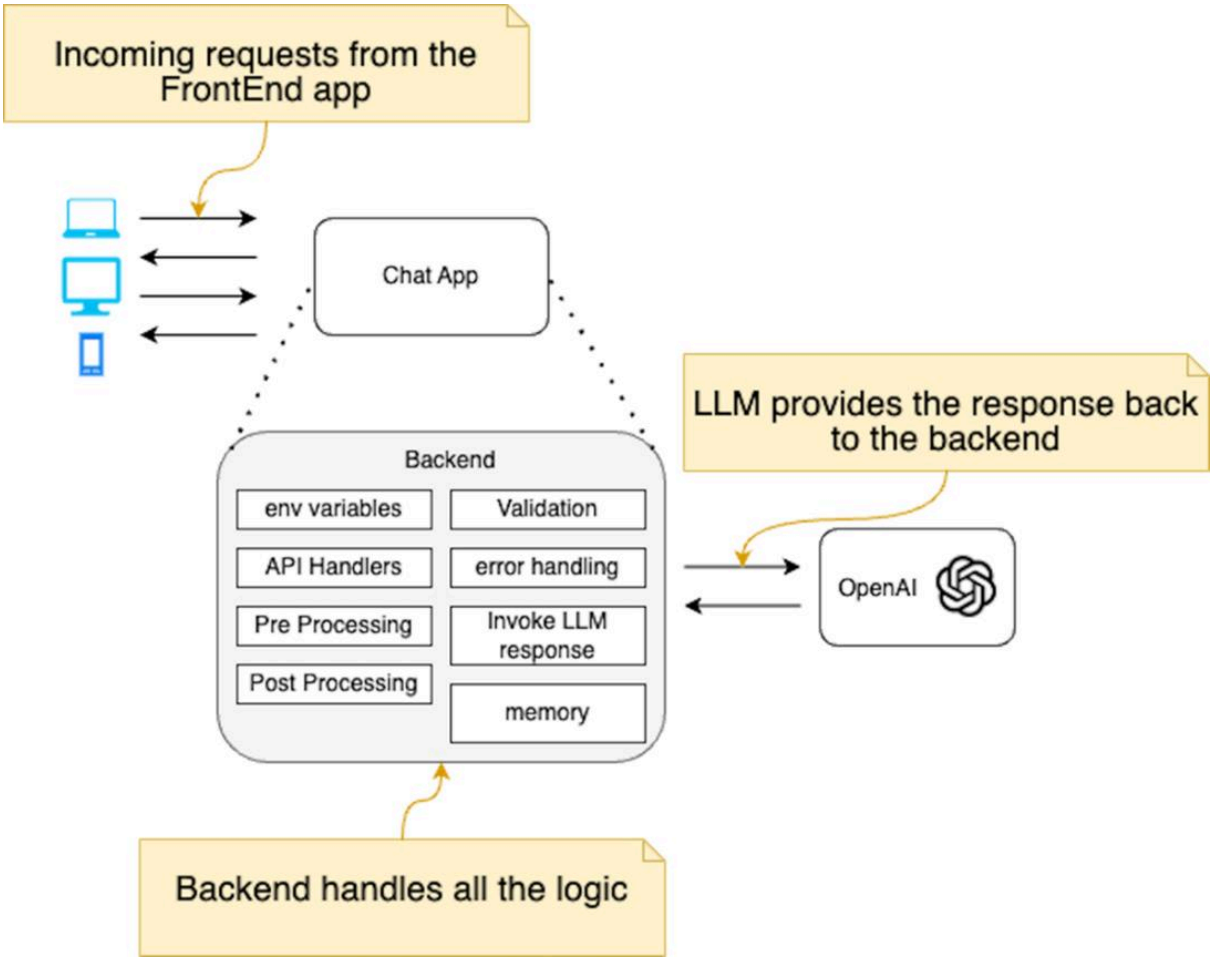
1. 사용자가 자신의 글을 입력합니다.
2. 화면에서 분석 요청을 누릅니다.
3. 실제 AI 자동 분석 대신 사람이 결과를 준비합니다.
4. 사용자는 결과를 확인하고 유용성에 대한 의견을 줍니다.
5. 반복 테스트를 통해 필요한 기능과 결과 형식을 수정합니다.

이 방식은 초기 개발 비용을 줄이면서도 사용자의 실제 반응을 확인할 수 있습니다. 다만 개인정보와 자료 보안을 지키고, 필요한 경우 테스트 방식에 대해 사용자에게 안내해야 합니다.

기성 AI 도구를 이용한 프로토타입도 가능합니다. 별도의 웹앱을 만들기 전에 기존 AI 도구로 핵심 사용 흐름을 실험하는 것입니다.

4. 아키텍처를 읽고 설계하는 방법

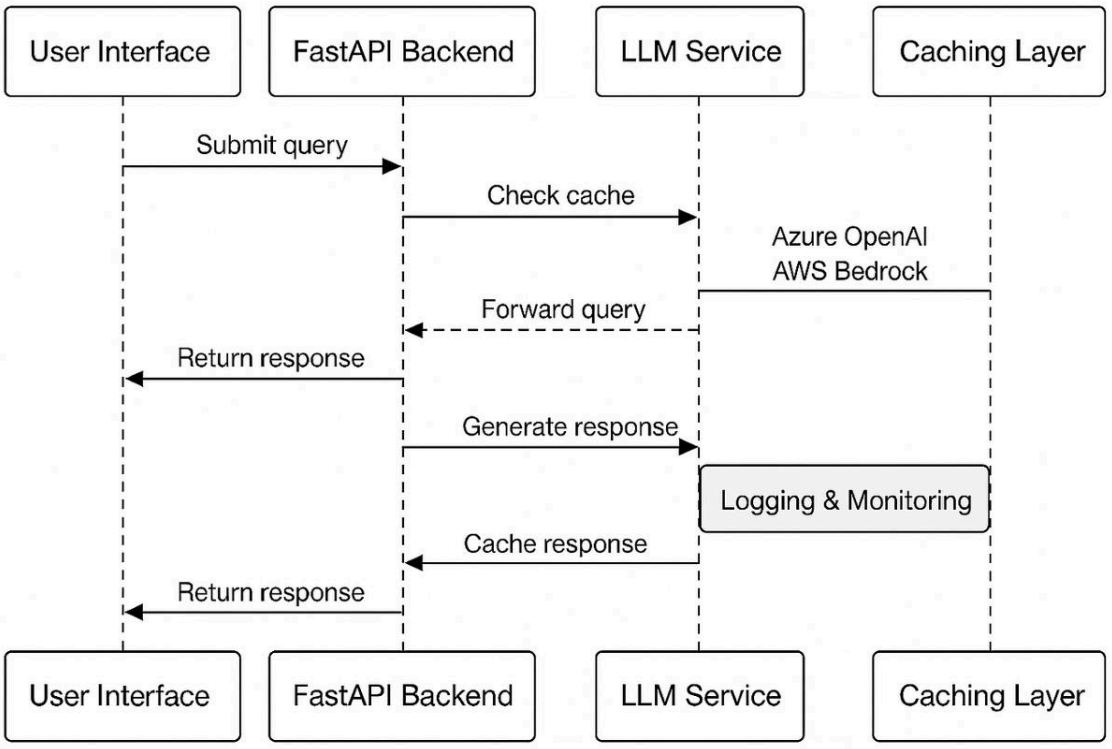
아키텍처는 서비스의 구성 요소와 연결 관계를 나타내는 설계도입니다. 세 가지 표현을 구분하면 이해하기 쉽습니다.



① 아키텍처 다이어그램 — 무엇이 연결되어 있는가?

사용자 화면, 서버, LLM, 데이터베이스 등 시스템의 주요 구성 요소와 관계를 보여 줍니다.

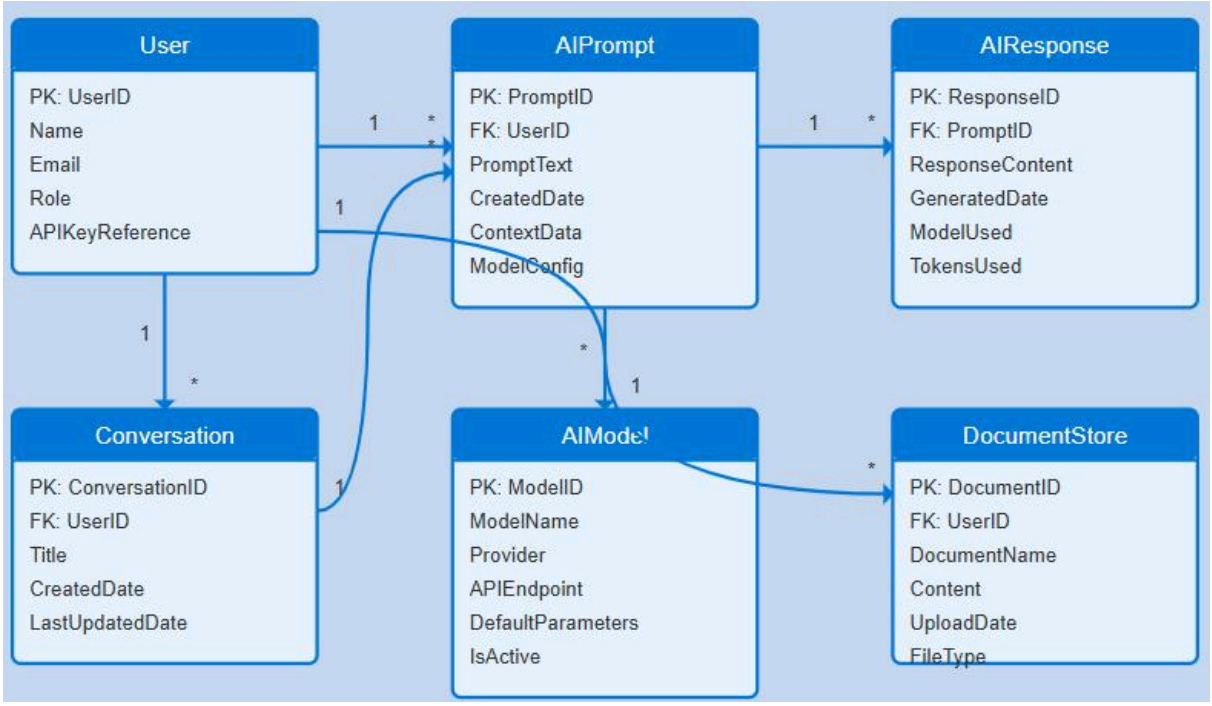
식당의 홀, 주방, 창고를 보여 주는 전체 배치도와 같습니다.



② 시퀀스 다이어그램 — 어떤 순서로 작동하는가?

사용자가 버튼을 누른 순간부터 서버 호출, AI 응답, 결과 저장, 화면 표시까지 시간 순서대로 표현합니다.

주문을 받은 뒤 조리하고 음식을 전달하는 순서도와 같습니다.



③ 데이터 모델 — 무엇을 저장하고 어떻게 연결하는가?

사용자, 프로젝트, 입력 문서, AI 분석 결과 등 데이터의 종류와 관계를 정의합니다.

어떤 정보를 어떤 서랍에 보관하고, 서랍끼리 어떻게 연결하는지 정하는 것과 같습니다.

예시: AI 글 분석 웹앱

사용자가 글을 입력하고 분석 버튼을 누르는 경우를 생각해 보겠습니다.

1. 사용자 화면

글 입력 · 분석 버튼

2. 서버

사용자 권한 · 입력 검증 · 비용 제한

3. LLM API

글 분석 및 결과 생성

4. 서버 및 데이터베이스

결과 검증 · 저장 · 사용자에게 반환

실제 시스템에서는 서버가 결과를 검증하고 데이터베이스에 저장하는 순서가 달라질 수 있으며, 저장이 필요 없는 서비스도 있습니다. 위 그림은 기본 구조를 이해하기 위한 예시입니다.

중요한 점은 API 키를 브라우저에 직접 노출하지 않고 서버에서 안전하게 관리하는 것입니다. 외부 공개 서비스라면 사용자 인증, 호출량 제한, 비용 추적도 함께 고려해야 합니다.

5. 실전 적용 예시: AI Story Partner(ASP)

1인 AI 컨설팅 사업을 준비하는 관점에서 보면, 이번 강의의 내용을 자신의 서비스에 바로 적용할 수 있습니다.

설계 항목	ASP 적용 예시
서비스 목적	고객의 업무와 아이디어를 분석해 AI 활용 방안을 제안
초기 사용자	AI 도입을 고민하는 소기업 대표 또는 실무자
제공 채널	기존 AI 도구로 검증한 뒤 필요하면 웹앱으로 확장
처리 방식	간단한 진단은 동기 처리, 여러 문서의 종합 분석은 비동기 처리 검토
오류 대응	결과 검증, 재시도, 호출 제한, 사람의 최종 확인
데이터 보호	고객 자료의 민감도 확인, 접근 권한 관리, 외부 전송 정책 명시
핵심 데이터	고객, 진단 요청, 입력 자료, 분석 결과, 검토 상태

설계 항목	ASP 적용 예시
검증 방법	소수 고객에게 프로토타입을 제공하고 사용성과 유용성을 확인

처음부터 모든 기능을 자동화할 필요는 없습니다. 초기에는 사람이 결과를 검토하는 방식으로 고객의 요구를 파악하고, 반복되는 업무부터 점진적으로 자동화하는 것이 현실적입니다.

6. AI 개발자가 알아야 하는 이유

AI 모델의 성능만으로는 서비스의 성공을 설명할 수 없습니다.

- 사용자가 접근하기 어렵다면 좋은 기능도 사용되지 않습니다.
- 보안과 접근 제어가 부족하면 조직에서 도입하기 어렵습니다.
- 처리 시간이 길면 사용자가 중간에 이탈할 수 있습니다.
- 비용 제한이 없으면 사용량 증가가 손실로 이어질 수 있습니다.
- 아키텍처가 불분명하면 기능을 수정하거나 확장하기 어렵습니다.
- 오류 상황을 고려하지 않으면 서비스 신뢰도가 떨어집니다.

따라서 AI 개발자는 모델과 프롬프트뿐 아니라 사용자 경험, 시스템 구조, 데이터 흐름, 운영 비용, 실패 대응까지 이해해야 합니다.

비개발자도 이러한 개념을 알면 코딩 에이전트에게 더 정확한 요구사항을 전달하고, 생성된 결과물이 실제 서비스에 적합한지 판단할 수 있습니다.

7. 학습 및 창업 목표와의 연결

이번 강의는 AI를 활용해 1인 사업을 구축하려는 학습자에게 특히 중요합니다.

첫째, 작게 검증하고 확장하는 사업 방식을 배울 수 있습니다. 큰 비용을 들여 웹앱을 만들기 전에 기존 AI 도구나 간단한 프로토타입으로 고객의 반응을 확인할 수 있습니다.

둘째, 고객에게 설명할 수 있는 설계 능력을 기를 수 있습니다. 고객이 AI를 도입할 때 필요한 보안, 비용, 데이터 처리, 운영 방식을 구조적으로 설명할 수 있습니다.

셋째, 코딩 에이전트를 효과적으로 활용할 수 있습니다. PRD와 아키텍처를 먼저 정리하면 에이전트가 무엇을 만들어야 하는지 명확해지고, 개발 중 방향이 바뀌는 문제도 줄어듭니다.

넷째, 반복 가능한 컨설팅 상품을 만들 수 있습니다. 고객의 요구를 분석하고, 적절한 제공 채널을 선택하며, 위험과 비용을 검토하는 과정을 표준화하면 AI 도입 진단 서비스로 발전시킬 수 있습니다.

8. 최종 산출물: 한 장짜리 PRD

PRD(Product Requirements Document)는 어떤 서비스를 왜, 누구를 위해, 어떻게 만들 것인지 정리한 제품 요구사항 문서입니다.

이번 챕터에서는 다음 항목을 포함하는 한 장짜리 PRD를 작성하는 것이 목표입니다.

- 서비스 목적: 해결하려는 문제와 제공 가치
- 대상 사용자와 사용 맥락: 누가 언제 어떤 상황에서 사용하는가
- 제공 채널과 처리 방식: 어떤 환경에서 어떤 방식으로 실행하는가
- 접근 제어와 데이터 처리: 누가 이용할 수 있으며 자료를 어떻게 관리하는가
- 오류 대응과 비용: 실패 시 대처 방법 및 사용량 제한
- 아키텍처: 화면, 서버, AI 모델, 데이터베이스의 연결 관계
- 파일럿 계획: 누구에게 어떤 방식으로 테스트하고 무엇을 측정할 것인가

좋은 PRD는 모든 기술적 세부 사항을 담는 문서가 아닙니다. 개발을 시작하기 전에 반드시 합의해야 하는 결정과 검증할 가설을 명확히 하는 문서입니다.

9. 반드시 기억할 내용

1. AI 기능과 AI 서비스는 다릅니다. 기능 구현 이후에도 사용성, 보안, 비용, 운영을 설계해야 합니다.
2. 가장 단순한 제공 방식부터 시작합니다. 기존 도구로 검증한 후 필요에 따라 웹앱으로 확장합니다.
3. 처리 시간에 따라 실행 방식을 결정합니다. 동기, 비동기, 스케줄 실행은 서로 다른 사용 상황에 적합합니다.
4. 실패를 전제로 설계합니다. 재시도, 시간 제한, 대체 경로, 검증, 비용 제한을 고려합니다.
5. 세 가지 설계도를 구분합니다. 아키텍처는 구성 요소, 시퀀스는 실행 순서, 데이터 모델은 저장 구조를 설명합니다.
6. PRD로 결정을 정리합니다. 요구사항이 명확해야 코딩 에이전트와 효율적으로 협업할 수 있습니다.
7. 배포보다 검증이 먼저입니다. 사용자가 실제로 가치를 느끼는지 확인한 후 개발 범위를 넓힙니다.

10. 한 문장 요약

AI 서비스를 잘 만든다는 것은 AI 기능을 구현하는 데 그치지 않고, 사용자가 안전하고 편리하게 지속적으로 이용할 수 있도록 제공 방식·처리 구조·오류 대응·데이터·비용을 함께 설계하는 것입니다.

Chapter02

강의정리 챕터 02. 사용 맥락 분석

1. 강의 목적

AI 기능을 설계할 때는 무엇을 만들 것인지보다 누가, 언제, 어떤 환경에서 사용하고, 결과를 받은 다음 무엇을 하는지를 먼저 파악해야 합니다.

아무리 유용한 AI 기능이라도 사용자가 평소 사용하는 도구와 분리되어 있거나, 필요한 순간에 접근하기 어렵다면 지속적으로 사용되지 않을 수 있습니다.

이번 강의에서는 다음을 학습합니다.

- 사용 맥락(Context of Use)을 분석하는 다섯 가지 항목
- 실제 사용자 인터뷰와 데스크 리서치 방법
- 사용자 요구에 맞는 AI 기능의 제공 채널 선택
- 동일한 AI 기능도 사용자 집단에 따라 설계가 달라지는 이유
- 코딩 에이전트를 활용한 인터뷰 질문 작성법

핵심은 기능 중심 사고에서 사용자 행동 중심 사고로 전환하는 것입니다.

2. 중학생도 이해할 수 있는 설명

2-1. 좋은 기능보다 중요한 것은 사용자의 생활 속에 들어가는 것입니다.

학교에서 공부할 때를 생각해 보겠습니다.

선생님이 강의 내용을 요약해 주는 AI를 만들었다고 가정하겠습니다. 그런데 요약 결과를 보려면 매번 별도 웹사이트에 접속해야 하고, 다음 날 휴대폰에서는 어제 결과를 찾을 수 없다면 어떨까요?

처음에는 신기해서 사용하더라도 점점 사용하지 않게 될 것입니다.

반면 평소 쓰는 노트 앱에 복습 노트가 저장되고, 다음 날 휴대폰에서 퀴즈를 바로 풀 수 있다면 계속 이용할 가능성이 커집니다.

즉, AI 기능의 가치는 AI가 얼마나 똑똑한지뿐 아니라 사용자의 기존 행동과 얼마나 자연스럽게 연결되는지에 달려 있습니다.

2-2. 사용 맥락을 분석하는 다섯 가지 항목

항목	확인할 질문
대상 사용자	누가 이 기능을 사용하는가?
필요한 시점	언제 이 기능이 필요한가?
기기와 도구	어떤 기기와 프로그램을 사용하는가?
다음 작업	결과를 받은 뒤 무엇을 하는가?
현재 방식	지금은 어떻게 처리하며 얼마나 걸리는가?

예를 들어 강의 복습 기능이라면 다음과 같습니다.

대상 사용자

강의를 듣고 직접 필기하는 수강생

필요한 시점

강의 직후 정리할 때, 다음 날 아침 복습할 때

기기와 도구

강의 직후에는 노트북과 노트 앱, 다음 날에는 휴대폰

다음 작업

복습 노트를 저장하고 다음 날 퀴즈 풀기

현재 방식

필기를 다시 읽으며 약 30분 동안 정리하지만, 자주 미룸

이 표를 채우면 어떤 서비스를 만들어야 하는지뿐 아니라, 어떤 방식으로 제공해야 하는지도 보이기 시작합니다.

3. 사용자 인터뷰와 데스크 리서치

3-1. 사용자 인터뷰: 상상이 아니라 실제 행동을 확인합니다.

사용자 인터뷰는 대상 사용자와 대화하면서 실제로 겪은 경험을 확인하는 방법입니다.

가장 중요한 원칙은 앞으로 사용할 의향을 묻기보다, 최근에 실제로 어떻게 행동했는지 묻는 것입니다.

좋지 않은 질문:

- "이런 AI 기능이 있으면 사용하시겠어요?"
- "복습을 자동으로 해 주면 좋겠죠?"

이러한 질문에는 상대방이 긍정적으로 대답할 가능성이 높습니다. 하지만 실제로 사용할지는 알 수 없습니다.

더 좋은 질문:

- "어제 강의가 끝난 뒤 필기를 어떻게 정리하셨나요?"
- "정리한 자료는 어디에 저장하셨나요?"
- "복습할 때 어떤 기기를 사용하셨나요?"
- "정리하는 데 얼마나 걸렸나요?"
- "마지막으로 복습을 미룬 이유는 무엇이었나요?"

인터뷰는 약 10분이면 시작할 수 있습니다. 장소, 기기, 도구, 결과물의 저장 위치, 소요 시간을 차례대로 확인하면 됩니다.

3-2. 데스크 리서치: 공개된 자료에서 사용자의 행동을 찾습니다.

직접 인터뷰하기 어렵다면 이미 공개된 사용자 경험을 조사할 수 있습니다.

- 유사한 노트 앱, 플래시카드 앱, AI 요약 서비스 찾기
- 앱 스토어와 서비스 후기에서 별점 2~3점 리뷰 살펴보기
- 커뮤니티에서 관련 질문과 불편 사항 검색하기
- 발견한 내용을 출처 링크와 함께 기록하기

별점 2~3점 리뷰는 서비스의 장점은 인정하면서도 실제 사용 중 불편했던 점을 설명하는 경우가 있어 유용합니다.

다만 리뷰 하나만으로 모든 사용자가 같은 문제를 겪는다고 판단해서는 안 됩니다. 서로 다른 두 곳 이상에서 유사한 내용이 확인되면 가정을 뒷받침하는 근거로 활용합니다.

3-3. 가정과 조사 결과를 구분합니다.

실제 사용자를 만나지 못했거나 충분한 근거가 없는 내용은 '가정'으로 표시합니다. 인터뷰나 리서치에서 확인된 내용은 근거와 출처를 함께 기록합니다.

구분	의미	예시
가정	아직 검증되지 않음	수강생은 휴대폰으로 퀴즈를 풀 것이다.
인터뷰	실제 사용자의 경험에서 확인	인터뷰한 수강생 3명 중 2명이 휴대폰으로 복습했다.

구분	의미	예시
리서치	공개된 자료에서 반복 확인	서로 다른 두 서비스의 리뷰에서 저장·동기화 불편이 확인됐다.

인터뷰 결과는 조사한 사람의 경험이지 전체 사용자의 통계가 아닙니다. 표본 수와 조사 출처를 함께 기록하면 판단의 신뢰도를 높일 수 있습니다.

4. 코딩 에이전트에게 맡기는 방법

코딩 에이전트는 코드를 작성하는 데만 사용하는 것이 아닙니다. 사용자 인터뷰를 준비하고 요구사항을 정리하는 데도 활용할 수 있습니다.

다음 프롬프트를 사용할 수 있습니다.

사용자 인터뷰 질문 생성 프롬프트

기능: 강의 필기를 입력하면 복습 노트와 퀴즈를 생성하는 기능

대상 사용자: 강의를 듣고 노트 앱에 필기하는 수강생

이 사용자가 강의 내용을 정리하고 복습하는 과정을 이해하기 위한 인터뷰 질문 5개를 만들어 주세요.

다음 사항을 확인할 수 있도록 구성해 주세요.

1. 최근에 이 작업을 수행한 시점과 장소
2. 당시 사용한 기기와 도구
3. 결과물을 저장하거나 공유한 위치
4. 결과를 받은 뒤 이어서 한 작업
5. 기존 방식과 소요 시간, 불편했던 점

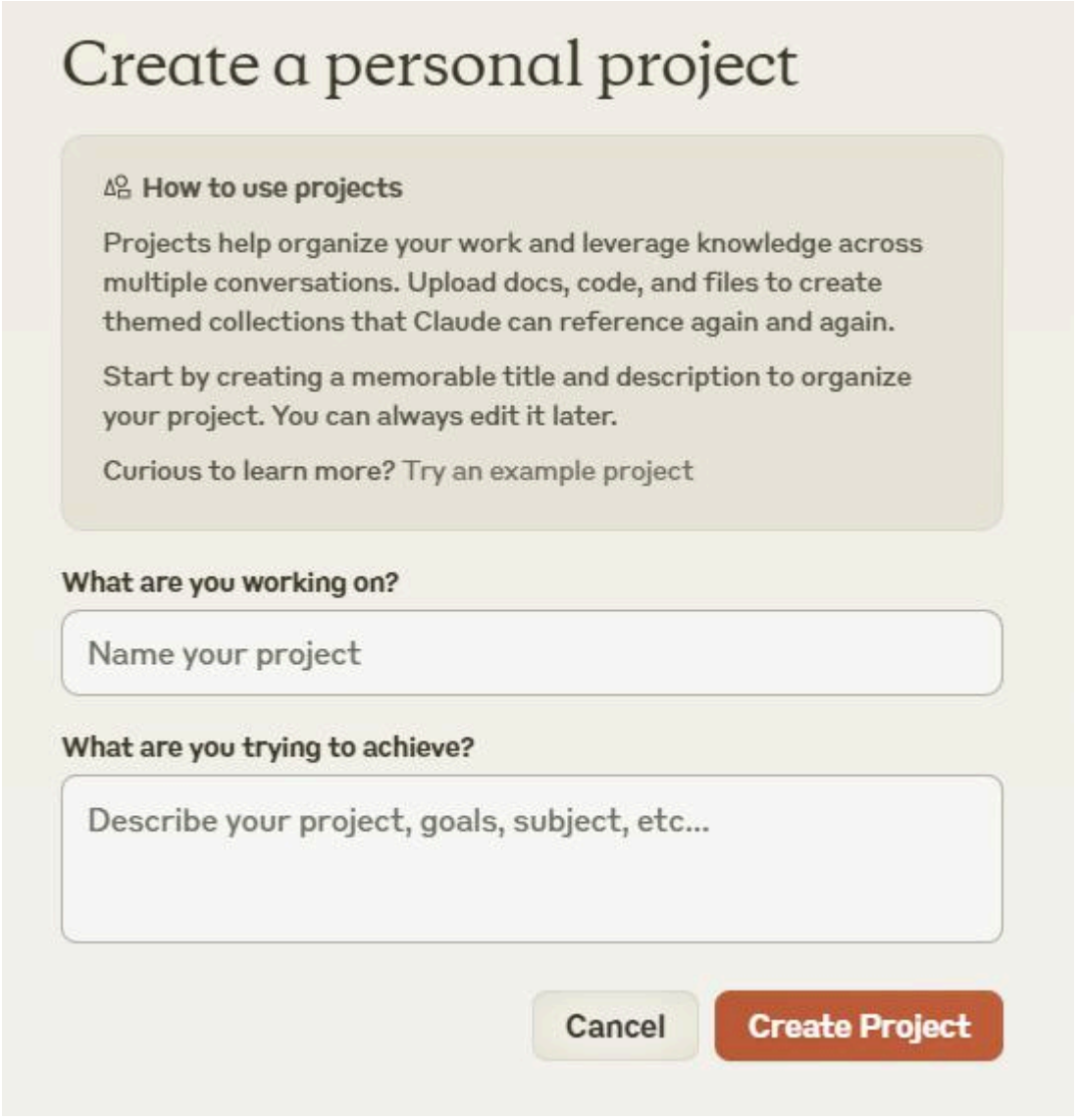
"이 기능을 사용하시겠어요?"처럼 가상의 의향을 묻는 질문은 제외하고, 최근에 실제로 한 행동을 묻는 질문으로 작성해 주세요.

사용자의 답변이나 사실을 임의로 만들어 내지 말고, 질문만 작성해 주세요.

이렇게 하면 에이전트가 사용자의 행동을 조사하기 위한 질문을 체계적으로 준비할 수 있습니다.

5. 다섯 가지 제공 채널

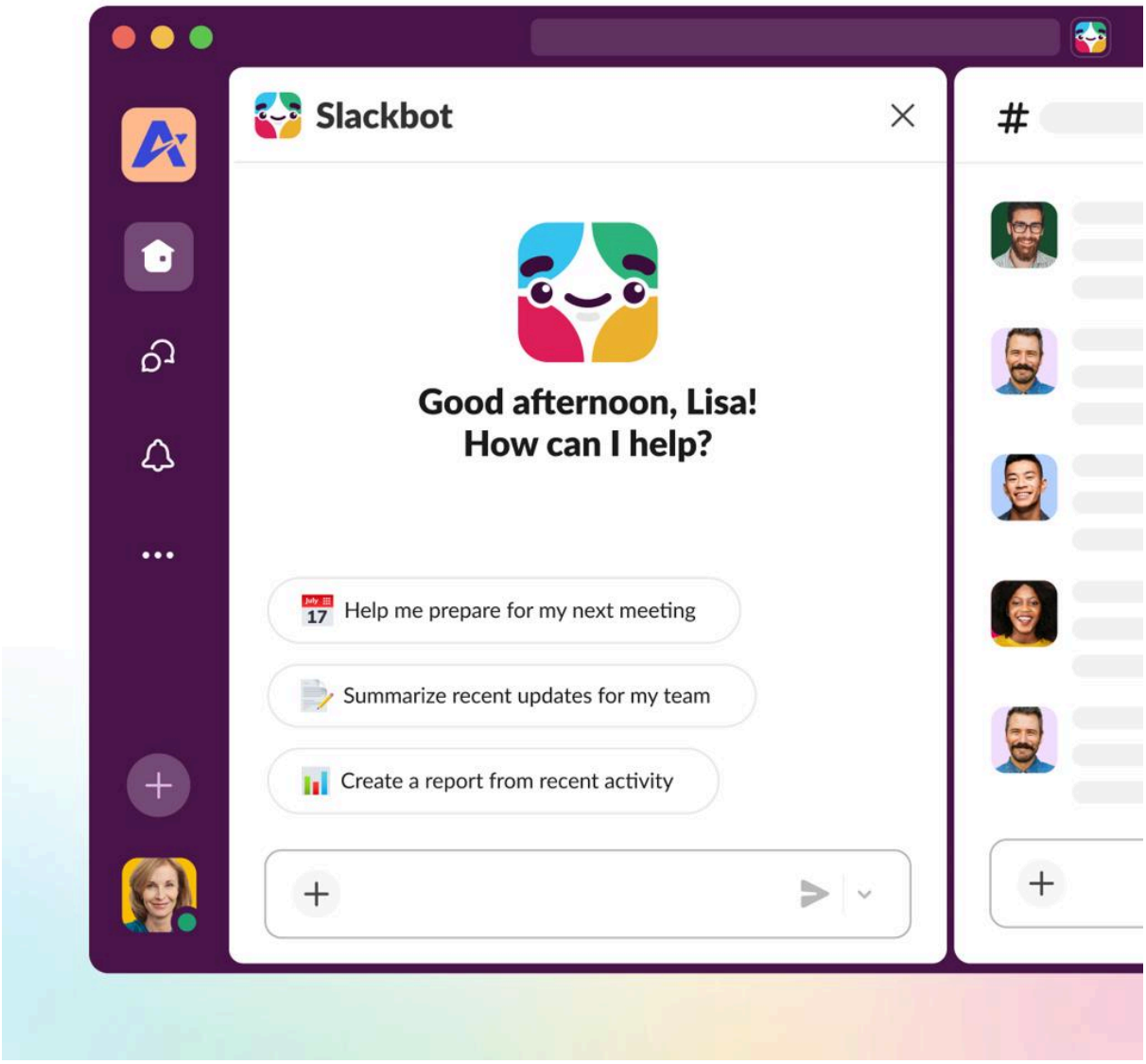
사용 맥락을 분석한 다음에는 AI 기능을 어떤 경로로 제공할지 결정합니다.



① 기성 AI 도구

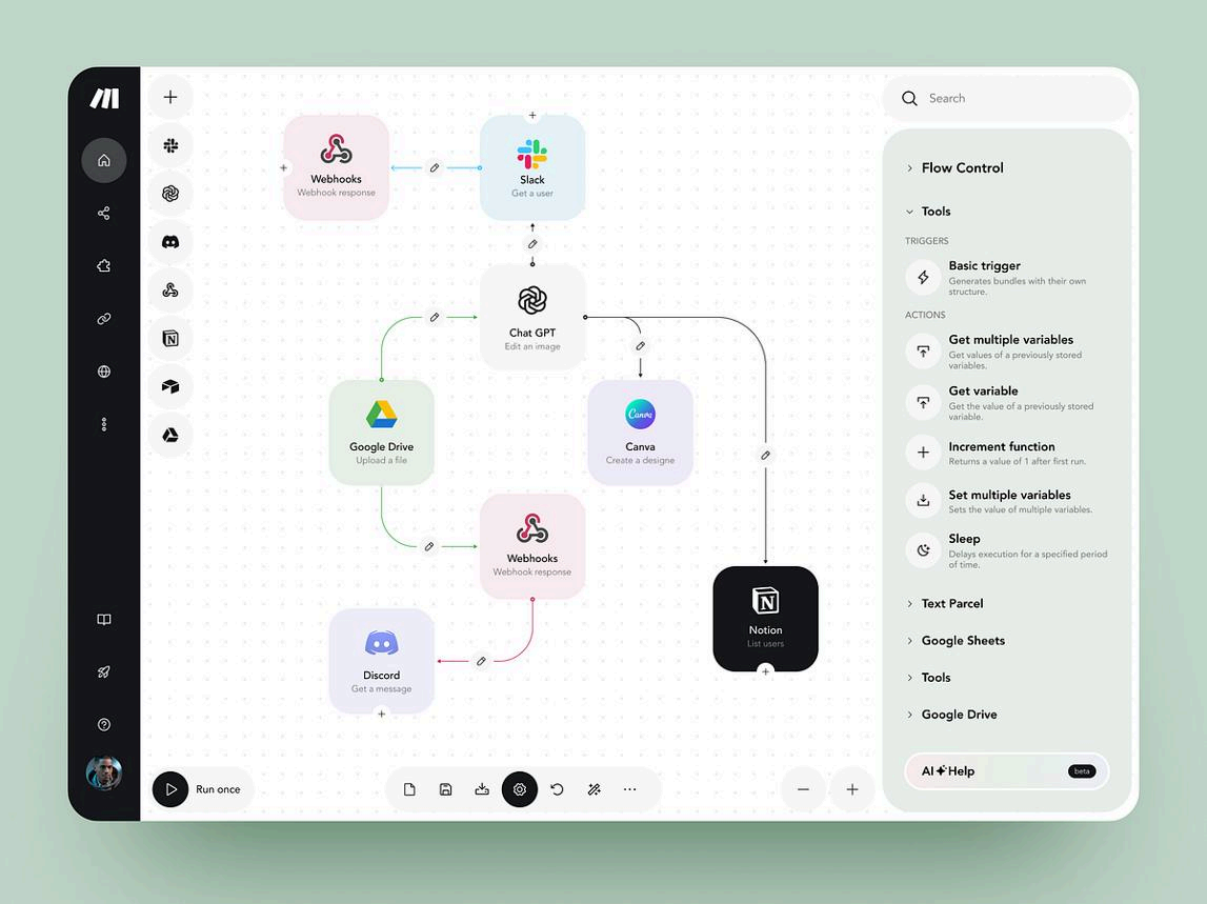
예: Claude Projects, GPTs. 기존 AI 도구에 지침과 참고 자료를 설정해 공유합니다.

가장 빠른 검증



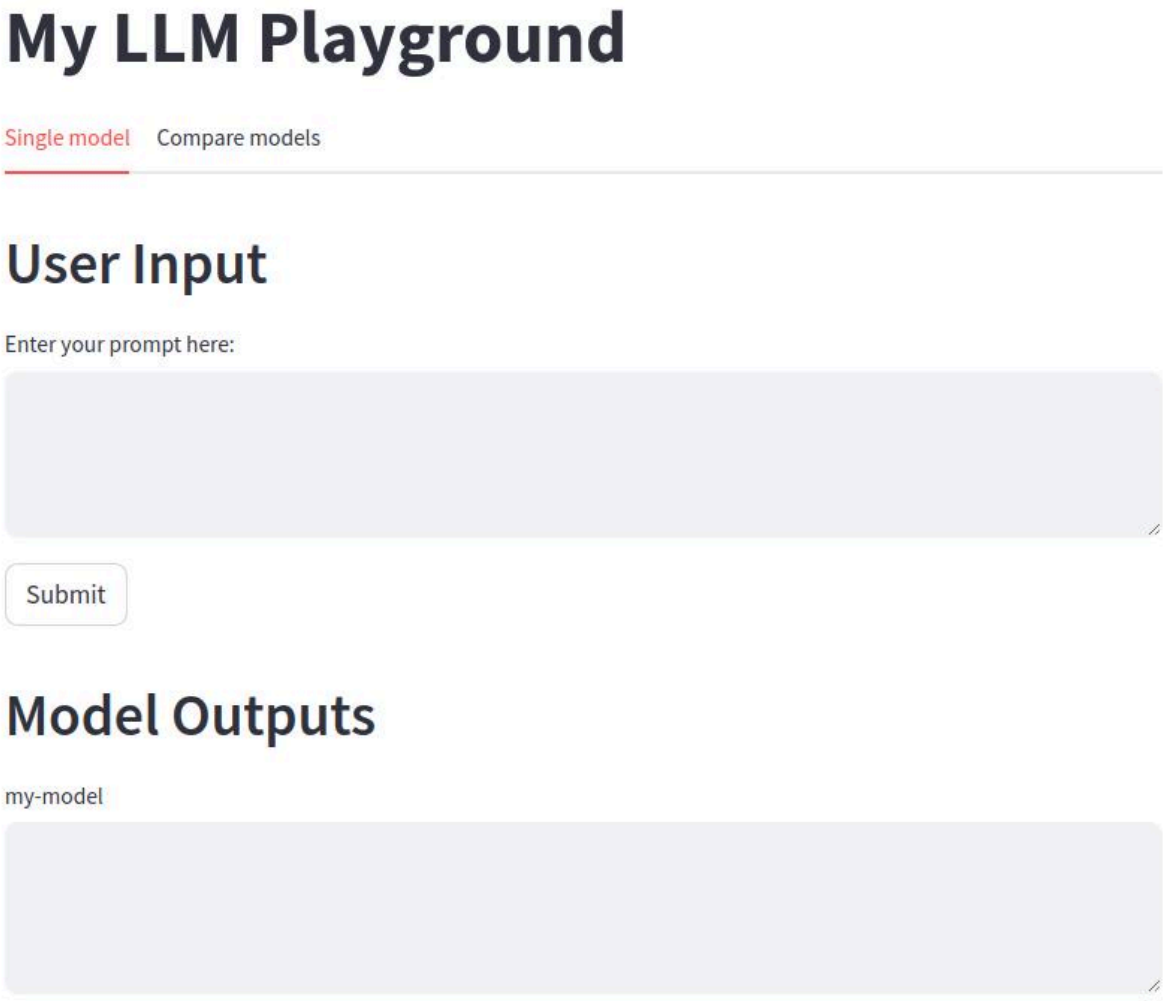
② 메신저 봇

예: 사내 메신저나 스터디 단체 대화방에서 AI를 호출합니다.



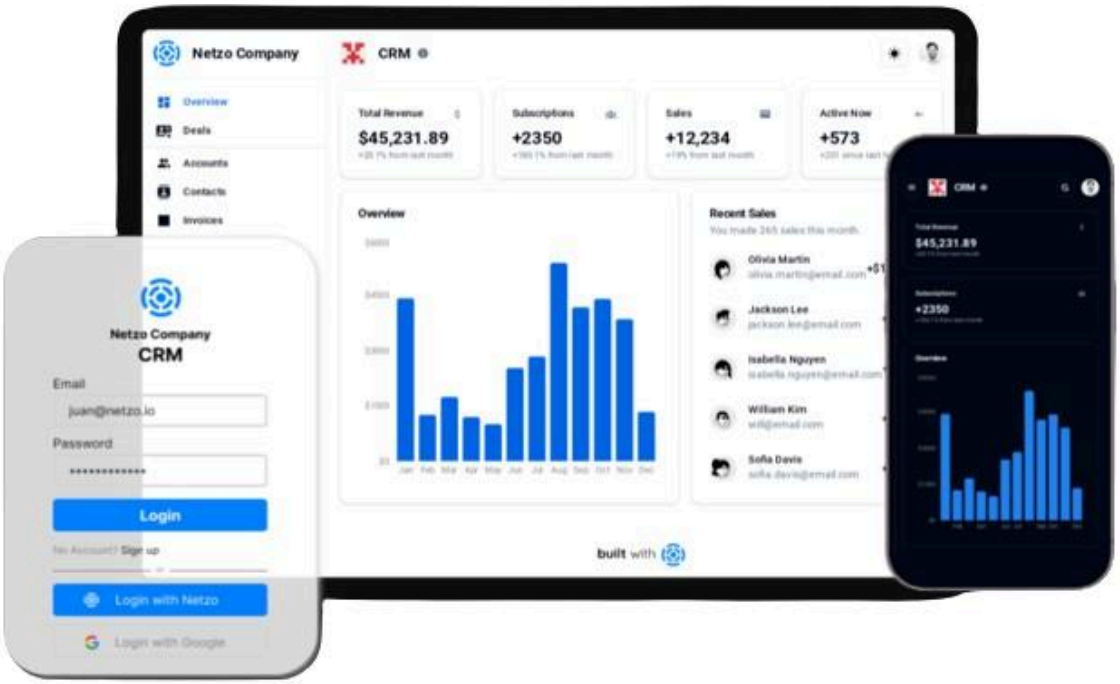
③ 자동화 워크플로

예: 정해진 시간에 퀴즈를 생성하고 메신저로 전송합니다.



④ 간단한 사용자 인터페이스

예: 입력창과 실행 버튼이 있는 간단한 도구를 제공합니다.



⑤ 독립형 웹앱

예: 회원가입, 개인별 데이터 저장, 권한, 결제 기능을 갖춘 서비스입니다.

제어 범위가 넓음

오른쪽으로 갈수록 화면과 데이터 저장, 접근 권한을 직접 제어하기 쉬워지지만 개발과 운영 부담도 커집니다.

처음부터 웹앱을 만들 필요가 없는 이유

기성 AI 도구로 먼저 검증하면 개발 부담을 줄일 수 있습니다. 이후 다음과 같은 요구가 생길 때 다른 채널로 옮길 수 있습니다.

- 기존 AI 도구의 계정이 없는 사용자도 이용해야 할 때
- 사용량과 비용을 직접 통제해야 할 때
- 메신저나 사내 시스템과 자동으로 연동해야 할 때
- 사용자별 데이터 저장이나 접근 권한을 세밀하게 관리해야 할 때

사용자가 실제로 필요로 한다는 증거를 확보한 다음, 필요한 만큼 개발하는 것이 합리적입니다.

6. 같은 AI 기능도 사용자에 따라 달라집니다.

강의 복습 노트와 퀴즈 생성 기능을 예로 들어 보겠습니다.

구분	개인 수강생	5명 스터디	외부 학습자
목적	혼자 복습	함께 학습	다수에게 서비스 제공
제공 채널 후보	기성 AI 도구	메신저 봇 또는 공유 도구	독립형 웹앱
주요 요구	간편한 사용과 저장	공동 이용과 정기 공유	회원 관리와 개인별 기록
처리 방식	즉시 생성	정해진 시간에 자동 실행 가능	긴 녹화는 비동기 처리 고려
주요 위험	자료 유실	자료 전송 동의와 공동 비용	보안, 저작권, 과금, 운영

여기서 중요한 점은 LLM이 수행하는 핵심 기능은 같다는 것입니다. 강의 내용을 분석해 복습 노트와 퀴즈를 만듭니다.

하지만 사용자가 누구인지, 어떤 목적으로 사용하는지에 따라 제공 채널, 데이터 처리, 비용, 접근 제어가 달라집니다.

특히 스터디 메신저 봇은 운영자 한 명의 API 키로 비용이 발생할 수 있으므로 호출 한도와 비용 분담을 정해야 합니다. 외부 학습자를 위한 웹앱은 자료의 권리와 이용 조건, 보존 기간, 개인정보 처리 등을 별도로 검토해야 합니다.

7. 실전 적용: 나의 AI 서비스에 적용하기

1인 AI 사업을 준비할 때는 아래 표를 직접 작성해 보시면 좋습니다.

사용 맥락 분석 실습

아래는 예시가 입력된 실습표입니다. 본인의 서비스에 맞게 수정해 보세요.

설계할 AI 기능

① 대상 사용자

자신의 경험과 글을 콘텐츠로 만들고 싶은 사람

② 필요한 시점

③ 사용 중인 기기와 도구

노트북, 브런치, 문서 도구

④ 결과를 받은 뒤의 다음 작업

글의 스타일을 검토하고 이야기로 발전

⑤ 현재 방식과 소요 시간

직접 읽고 수정하며 콘텐츠 방향을 결정

정보의 근거

아직 확인하지 못한 가정

실제 사용자 인터뷰에서 확인

공개 자료 조사에서 확인

분석 결과 검토하기

이 표를 작성한 뒤에는 대상 사용자와 가까운 사람 한 명을 인터뷰하고, 공개된 후기와 커뮤니티 글을 확인해 가정을 줄이는 것이 다음 단계입니다.

8. 객관식 퀴즈

문제: 스터디원 5명이 강의 자료를 질문하면 해당 슬라이드 번호와 함께 답해 주는 기능을 원합니다. 5명 모두 같은 AI 도구의 유료 계정이 있고, 강의 자료는 PDF 20개입니다. 가장 먼저 시도할 제공 채널은 무엇일까요?

① 회원가입과 관리자 화면이 있는 웹앱을 개발한다.

② 기성 AI 도구의 프로젝트에 강의 자료와 응답 규칙을 설정해 공유한다.

③ 강의 플랫폼에 브라우저 확장 프로그램을 개발한다.

④ 모든 기수가 사용할 메신저 봇을 개발한다.

정답 확인

9. 반드시 기억할 내용

1. 기능보다 사용 맥락을 먼저 분석합니다. 누가, 언제, 어떤 도구를 사용하고 결과를 어떻게 활용하는지 확인합니다.
2. 사용자의 의향보다 실제 행동을 조사합니다. 최근 경험을 묻는 인터뷰가 더 유용합니다.
3. 가정과 근거를 구분합니다. 인터뷰와 리서치 결과를 출처와 함께 기록합니다.
4. 제공 채널은 사용자가 결정합니다. 개인, 스터디, 외부 고객은 같은 기능에도 서로 다른 요구가 있습니다.
5. 가장 단순한 방법으로 먼저 검증합니다. 처음부터 웹앱을 만들 필요는 없습니다.
6. 결과를 받은 뒤의 다음 작업까지 설계합니다. AI 결과가 사용자의 기존 업무 흐름과 자연스럽게 연결되어야 합니다.
7. 코딩 에이전트는 조사 준비에도 활용합니다. 질문 작성과 요구사항 정리를 맡기되, 실제 사용자 답변을 만들어 내게 해서는 안 됩니다.

10. 한 문장 요약

좋은 AI 서비스는 만들고 싶은 기능에서 출발하는 것이 아니라, 사용자가 실제로 일하고 생활하는 맥락을 이해하고 그 흐름에 가장 자연스럽게 연결되는 방식으로 설계하는 것입니다.

Chapter03

강의정리 챕터 03. 동기 처리, 비동기 처리, 스케줄 실행

1. 강의 목적

AI 기능을 서비스로 제공하려면 AI가 어떤 일을 하는지뿐 아니라 얼마나 오래 걸리는지, 사용자가 언제 결과를 필요로 하는지, 오류가 발생했을 때 어떻게 대응할지를 설계해야 합니다.

일반적인 웹 서비스는 짧은 시간 안에 응답하지만, LLM은 몇 초에서 몇 분까지 걸릴 수 있습니다. 모든 요청을 같은 방식으로 처리하면 사용자가 오래 기다리거나, 작업이 중단되거나, 잘못된 결과를 그대로 받아 보는 문제가 발생할 수 있습니다.

이번 챕터에서는 다음을 학습합니다.

- 동기 처리, 비동기 처리, 스케줄 실행의 차이와 선택 기준
- 사용자의 다음 작업에 맞는 AI 출력 형식 설계
- AI 오류에 대응하는 다섯 가지 안전장치
- 오류의 영향도에 따른 대응 수준 결정
- 사용자에게 표시할 상태 메시지와 오류 안내 작성

핵심은 AI가 결과를 생성하는 과정뿐 아니라, 사용자가 결과를 기다리고 확인하고 활용하는 전체 경험을 설계하는 것입니다.

2. 중학생도 이해할 수 있는 설명

2-1. AI에게 일을 맡기는 세 가지 방법

식당에 음식을 주문하는 상황으로 비유해 보겠습니다.



① 동기 처리 — 끝날 때까지 기다리기

음료처럼 금방 나오는 주문입니다. 주문한 뒤 결과가 나올 때까지 기다렸다가 바로 받습니다.

AI 예시: 짧은 글 요약, 간단한 번역, 한두 문장의 질문과 답변



② 비동기 처리 — 접수하고 나중에 결과 받기

시간이 오래 걸리는 요리를 주문하고 접수 번호를 받습니다. 다른 일을 하다가 완성되면 알림을 받습니다.

AI 예시: 긴 녹음 요약, 여러 문서 분석, 대량 콘텐츠 생성



③ 스케줄 실행 — 정해진 시간에 자동으로 처리하기

매주 월요일 오전에 단체 도시락을 준비하도록 예약하는 것과 같습니다.

AI 예시: 매일 아침 뉴스레터 생성, 매주 시장 분석 보고서 작성

세 방식의 차이는 단순히 AI가 빠른지 느린지에만 있지 않습니다. 사용자가 직접 요청하는지, 얼마나 기다릴 수 있는지, 결과가 언제 필요한지도 중요합니다.

3. 세 가지 처리 방식의 차이

구분	동기 처리	비동기 처리	스케줄 실행
실행 시점	사용자가 요청할 때	사용자가 요청할 때	정해진 시간·주기에 자동 실행
사용자 대기	결과가 나올 때까지 기다림	접수 후 다른 일을 할 수 있음	실행 전까지 기다릴 필요 없음
결과 전달	즉시 응답	상태 조회 또는 완료 알림	예약된 실행 결과를 전달
적합한 작업	짧고 간단한 작업	오래 걸리는 작업	반복적이고 정기적인 작업
주요 고려 사항	시간 초과, 연결 끊김	작업 상태, 재시도, 알림	실행 주기, 중복 실행, 실패 감지

3-1. 동기 처리

사용자가 요청하면 서버가 AI 처리를 진행하고, 결과를 받은 뒤 사용자에게 응답합니다.

예를 들어 필기 한 페이지를 요약하는 작업이라면 몇 초 안에 결과를 제공할 수 있으므로 동기 처리가 적합할 수 있습니다.

다만 동기 처리라고 해서 반드시 빠른 것은 아닙니다. 응답 시간이 길어지면 사용자가 기다려야 하고, 서버나 연결의 제한 시간에 걸릴 수도 있습니다.

3-2. 비동기 처리

비동기 처리는 오래 걸리는 작업을 접수하고 실제 처리를 별도로 진행하는 방식입니다.

일반적인 흐름은 다음과 같습니다.

1. 사용자가 녹음 파일 업로드

2. 작업 큐에 처리 요청 등록

3. 접수 번호와 상태 안내

"분석 요청이 접수되었습니다."

4. 백그라운드 작업자가 AI 분석

5. 결과 저장 및 완료 알림

여기서 작업 큐(Queue)는 처리할 작업을 차례로 보관하는 대기 목록이고, 워커(Worker)는 그 목록에서 작업을 가져와 실제로 실행하는 프로그램입니다.

비동기 처리에서는 다음 사항도 설계해야 합니다.

- 사용자가 작업 상태를 확인할 수 있는가?
- 실패한 작업을 다시 실행할 수 있는가?
- 같은 요청이 중복 처리되지 않도록 할 것인가?
- 완료된 결과를 어디에 저장하고 어떻게 알릴 것인가?

비동기 방식이라고 해서 실패가 사라지는 것은 아닙니다. 작업 상태와 복구 방법까지 마련해야 안정적인 서비스가 됩니다.

3-3. 스케줄 실행

스케줄 실행은 사용자가 매번 직접 요청하지 않아도 정해진 시간에 작업을 시작하는 방식입니다.

예를 들어 AI 뉴스레터 에이전트가 매일 오전 7시 30분에 뉴스를 수집하고, 평가하고, 요약해 발행 초안을 만든다고 가정해 보겠습니다.

사용자는 매일 버튼을 누르지 않아도 됩니다.

다만 스케줄 실행과 비동기 처리는 서로 배타적인 개념이 아닙니다. 정해진 시간에 작업을 시작한 뒤, 실제 분석은 비동기로 진행할 수도 있습니다.

예를 들어 오전 8시에 주간 보고서 작성을 시작하고, 작업 큐에서 여러 문서를 분석한 뒤 완료 알림을 보내도록 설계할 수 있습니다.

4. 출력 형식도 함께 설계해야 합니다

AI에게 단순히 "요약해 줘"라고 요청하면 결과의 길이와 구성, 표현 방식이 매번 달라질 수 있습니다.

따라서 출력 형식은 사용자가 결과를 받은 뒤 수행할 작업에 맞춰 미리 정해야 합니다.

예를 들어 다음 날 휴대폰으로 퀴즈를 푸는 것이 목적이라면 다음과 같이 설계할 수 있습니다.

항목	설계 예시
문제 수	5개
문제 형식	객관식 4지선다
문제 길이	휴대폰에서 한눈에 읽을 수 있는 길이
정답 위치	문제와 분리해 표시
해설	정답의 이유를 1~2문장으로 설명
출처	관련 강의 자료의 페이지 또는 문단 표시
정보가 없을 때	임의로 만들지 않고 "자료에서 확인되지 않음"으로 표시

이러한 형식을 프롬프트에 명시하면 결과의 일관성을 높일 수 있습니다.

웹앱으로 구현할 때는 출력 형식을 응답 스키마(Response Schema)로 정의할 수도 있습니다. 예를 들어 문제, 선택지, 정답, 해설, 출처를 각각 별도의 필드로 구분하면 화면에 표시하거나 데이터베이스에 저장하기 쉬워집니다.

핵심은 AI가 무엇을 생성할지만 정하지 말고, 결과가 어떤 형태여야 다음 작업에 바로 사용할 수 있는지도 정하는 것입니다.

5. AI 오류 대응: 다섯 가지 안전장치

LLM은 그럴듯하지만 틀린 답을 생성할 수 있습니다. 강의 자료에 없는 설명을 덧붙이거나, 퀴즈의 정답을 잘못 표시할 수도 있습니다.

따라서 오류가 발생하지 않는다고 가정하기보다, 오류가 사용자에게 미치는 영향을 줄이는 방법을 설계해야 합니다.

① 출처 표시

AI 답변이 어떤 자료를 근거로 만들어졌는지 보여 줍니다.

예: "이 설명은 강의 자료 12페이지를 근거로 작성했습니다."

② 휴먼 인 더 루프(Human-in-the-Loop, HITL)

AI 결과를 사람에게 보여 주고 승인받은 뒤 외부에 전달하거나 실행합니다.

예: 스터디 운영자가 퀴즈의 정답을 확인한 뒤 단체방에 게시합니다.

③ 폴백 응답(Fallback)

AI가 정상적으로 답하지 못할 때 안전한 대체 응답이나 다른 처리 경로를 제공합니다.

예: "자료에서 답을 확인하지 못했습니다. 원문을 확인해 주세요."

④ 에스컬레이션(Escalation)

자동으로 해결하기 어려운 문제를 담당자나 전문가에게 전달합니다.

예: 정답이 모호하거나 근거가 충돌하는 문제는 강사에게 검토를 요청합니다.

⑤ 피드백과 되돌리기

사용자가 오류를 신고하거나 이전 결과를 수정·취소할 수 있게 합니다.

예: "정답이 틀렸어요" 버튼을 제공하고, 수정된 퀴즈를 다시 게시할 수 있게 합니다.

오류의 영향도에 따라 장치를 선택합니다.

모든 서비스에 다섯 가지 장치를 전부 적용할 필요는 없습니다.

상황	영향도 예시	적절한 대응
혼자 보는 복습 노트에 작은 설명 오류	낮음~중간	출처 표시, 수정·피드백
여러 사람에게 공유하는 퀴즈의 정답 오류	중간~높음	출처 표시, 사람의 검토, 정답 불확실 시 담당자 확인
고객에게 잘못된 결과를 자동 발송	높음	발송 전 승인, 실패 시 발송 중단, 담당자 에스컬레이션
AI 호출 실패로 결과가 나오지 않음	상황에 따라 다름	폴백 응답, 재시도, 사용자 안내

오류 대응 장치가 많아질수록 안전성이 높아질 수 있지만, 사용자가 거쳐야 하는 단계와 운영 부담도 늘어납니다. 따라서 오류의 발생 가능성과 피해 규모를 고려해 필요한 수준의 안전장치를 선택해야 합니다.

피드백을 수집할 때는 사용자의 평가를 그대로 정답으로 간주하지 않고, 검토된 자료와 함께 활용해야 합니다. 이렇게 모은 데이터는 이후 AI 응답 품질을 평가하고 개선하는 데 도움이

됩니다.

6. 실전 적용: 나의 AI 서비스 설계하기

1인 AI 사업이나 AI 스토리 코치 같은 서비스에 적용한다면 다음 세 가지를 정리하면 됩니다.

6-1. 처리 방식 결정

기능	추천 방식	이유
짧은 글의 스타일 분석	동기 처리	사용자가 즉시 결과를 확인
긴 원고 여러 편의 종합 분석	비동기 처리	처리 시간이 길고 진행 상태가 필요
매주 콘텐츠 아이디어 보고서 생성	스케줄 실행	정기적인 반복 업무를 자동화

실제 선택은 원고 길이, 처리 시간, 사용자의 대기 가능 시간에 따라 달라집니다.

6-2. 사용자에게 보이는 흐름 작성

예를 들어 긴 원고를 분석하는 서비스라면 다음과 같습니다.

1. 사용자가 원고를 업로드합니다.
2. 화면에 "분석 요청이 접수되었습니다"라고 표시합니다.
3. 분석 중에는 진행 상태를 확인할 수 있습니다.
4. 완료되면 웹 화면이나 이메일로 알립니다.
5. 결과에 출처와 분석 근거를 표시합니다.
6. 사용자가 오류를 신고하거나 결과를 수정할 수 있게 합니다.

6-3. 오류 대응표 작성

오류 유형	영향도	대응 장치	사용자에게 보이는 문구
원고에 없는 내용을 추가함	중간	출처 표시, 폴백	"원문에서 근거를 찾지 못한 내용은 분석 결과에서 제외했습니다."
분석 중 API 호출 실패	중간	제한된 재시도, 폴백	"분석 중 문제가 발생했습니다. 잠시 후 다시 시도해 주세요."
파일이 너무 크거나 형식이 맞지 않음	중간	입력 검증	"현재 파일 형식을 지원하지 않습니다. PDF 또는 DOCX 파일을 업로드해 주세요."
결과의 핵심 해석이 불확실함	높음일 수 있음	사람의 검토, 에스컬레이션	"일부 해석의 근거가 충분하지 않아 확인이 필요합니다."

사용자에게 보여 주는 문구는 단순히 오류가 발생했다는 사실만 전달하는 것이 아니라, 현재 상태와 다음에 할 일을 함께 알려 주는 것이 좋습니다.

7. AI 개발자와 1인 창업가에게 중요한 이유

이 챕터의 내용은 단순한 기술 선택을 넘어 서비스의 신뢰성과 운영 효율을 결정합니다.

- 사용 경험: 불필요하게 오래 기다리지 않도록 합니다.
- 운영 안정성: 연결이 끊기거나 API 호출이 실패해도 작업을 복구할 수 있습니다.
- 결과 품질: 출력 형식과 출처를 명확히 해 결과를 검토하기 쉽게 만듭니다.
- 위험 관리: 잘못된 답변이 여러 사람에게 전달되거나 외부 행동으로 이어지는 일을 줄입니다.
- 비용 관리: 재시도 횟수와 실행 주기를 조절해 불필요한 호출을 줄입니다.
- 지속적 개선: 사용자 피드백과 검토 결과를 축적해 품질을 개선합니다.

코딩 에이전트에게 "긴 녹음을 요약하는 기능을 만들어 주세요"라고 요청하는 것만으로는 부족합니다.

"접수 후 비동기로 처리하고, 상태를 표시하며, 실패 시 제한된 횟수로 재시도하고, 완료되면 알림을 보내며, 출처가 없는 설명은 확정적으로 제시하지 않는다"처럼 요구사항을 구체적으로 전달해야 합니다.

8. 반드시 기억할 내용

1. 동기 처리: 짧은 작업을 요청 즉시 처리하고 결과를 반환합니다.
2. 비동기 처리: 오래 걸리는 작업을 접수한 뒤 별도로 처리하고 상태와 결과를 전달합니다.
3. 스케줄 실행: 정해진 시간이나 주기에 따라 자동으로 작업을 시작합니다.
4. 출력 형식: 사용자의 다음 작업에 맞춰 항목, 순서, 길이, 출처, 값이 없을 때의 표현을 정의합니다.
5. 오류 대응: 출처 표시, HITL, 폴백, 에스컬레이션, 피드백과 되돌리기를 상황에 맞게 조합합니다.
6. 영향도 중심 설계: 오류의 피해가 클수록 더 강한 검증과 승인 절차가 필요합니다.
7. 사용자 메시지: 오류 사실뿐 아니라 현재 상태와 다음 행동을 알려 줍니다.

9. 객관식 퀴즈

문제: 외부 사용자가 1시간짜리 강의 녹음을 업로드하면 요약을 생성하는 웹앱입니다. 처리에는 3~5분이 걸립니다. 가장 알맞은 처리 방식은 무엇일까요?

① 처리가 끝날 때까지 로딩 화면을 띄우고 동기로 응답한다.

② 업로드 직후 접수 응답을 주고 작업 큐에서 처리한 뒤, 상태를 보여 주고 완료되면 메일로 알린다.

③ 매일 오전 8시에 그동안 업로드된 녹음을 배치로 처리한다.

④ 처리 시간을 줄이기 위해 녹음의 앞 10분만 요약한다.

정답 확인

10. 한 문장 요약

AI 서비스를 설계할 때는 작업 시간과 결과가 필요한 시점에 맞춰 처리 방식을 선택하고, 사용자가 활용하기 좋은 출력 형식과 오류 대응 장치를 함께 설계해야 합니다.

Chapter04

/강의정리 챕터 04. 코드 없이 검증하기

1. 강의의 목적

이번 강의의 핵심은 AI 서비스를 개발하기 전에, 실제 사용자가 원하는 방식으로 사용할 수 있는지 먼저 확인하는 것입니다.

앞선 강의에서 우리는 다음 사항을 정했습니다.

- 제공 채널: 어디에서 사용할 것인가?
- 처리 방식: 즉시 처리할 것인가, 나중에 처리할 것인가?
- 출력 형식: 어떤 결과를 어떤 형태로 보여 줄 것인가?
- 오류 대응: AI가 잘못된 결과를 내거나 자료가 부족할 때 어떻게 할 것인가?

하지만 이것들은 아직 가정입니다. 사용자가 실제로 써 보기 전까지는 맞다고 확신할 수 없습니다.

따라서 코드를 작성하기 전에 간단한 시제품을 만들어 사용자의 행동을 관찰하고, 문제를 발견하면 설계를 수정해야 합니다.

2. 중학생도 이해할 수 있는 설명

새로운 식당을 연다고 생각해 보겠습니다.

인테리어와 주방을 완성하고 메뉴를 모두 개발한 다음에야 손님에게 음식을 제공한다면, 손님들이 메뉴를 이해하지 못하거나 주문 방식이 불편하다는 사실을 뒤늦게 발견할 수 있습니다.

반대로 정식 개점 전에 몇 명에게 임시 메뉴판을 보여 주고 음식을 제공하면, 적은 비용으로 문제를 발견할 수 있습니다.

AI 서비스도 마찬가지입니다.

1단계. 가설 세우기

사용자가 어떤 채널에서 어떤 결과를 원할지 정합니다.

2단계. 코드 없이 시험하기

사람이 AI 역할을 하거나, 기존 AI 도구에 지침을 넣어 시제품을 만듭니다.

3단계. 실제 행동 관찰하기

사용자가 어디에서 멈추고, 결과를 어떻게 수정하는지 확인합니다.

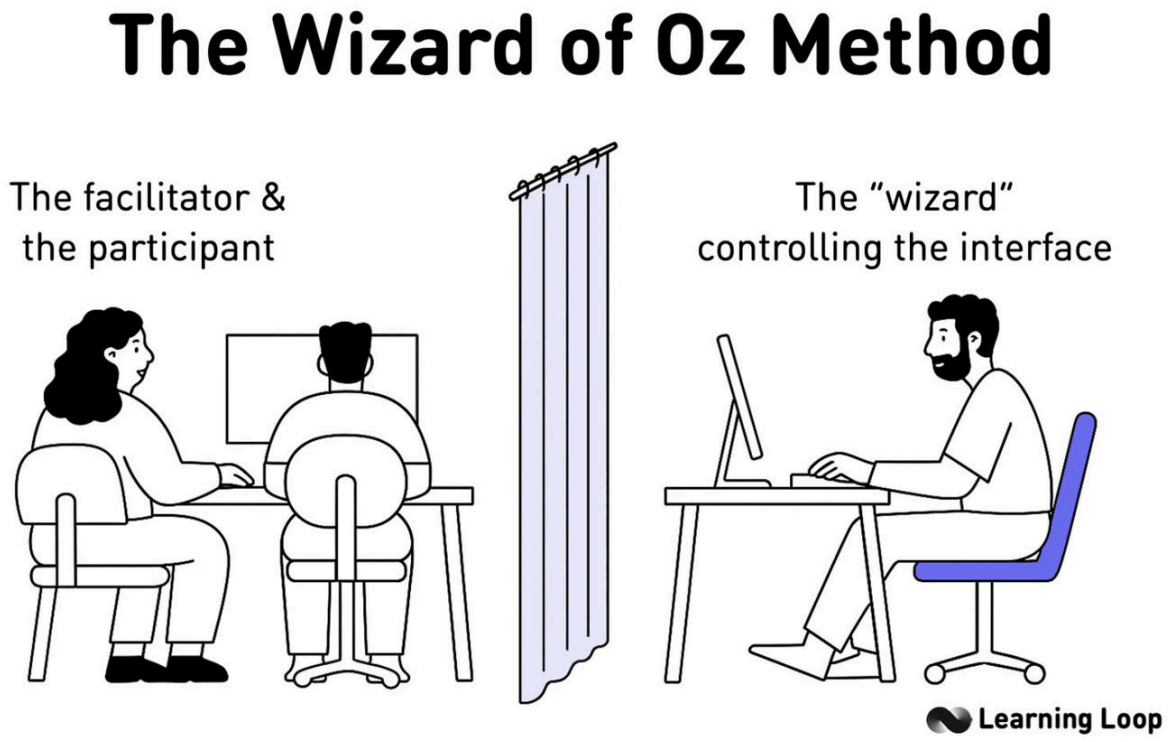
4단계. 설계 수정하기

발견한 문제에 따라 채널, 출력 형식, 안내 문구, 오류 대응 등을 바꿉니다.

핵심은 완성도 높은 제품을 먼저 만드는 것이 아니라, 올바른 제품을 만들기 위해 먼저 확인하는 것입니다.

3. 핵심 개념 두 가지

3-1. 오즈의 마법사 테스트(Wizard of Oz Prototyping)



learningloop.io

실제 시스템이 자동으로 처리하는 것처럼 보이지만, 뒤에서는 사람이 대신 처리하는 테스트 방법입니다.

예를 들어 메신저 복습봇을 만든다고 가정하겠습니다.

- 사용자는 복습봇에 강의 필기를 보냅니다.
- 진행자는 봇처럼 행동하며 필기를 읽습니다.
- 진행자가 미리 정한 출력 형식에 따라 복습 자료를 작성합니다.
- 사용자는 실제 봇을 사용하는 것처럼 결과를 확인합니다.

이 방법으로 AI 모델을 만들지 않고도 사용자가 채널과 사용 흐름을 이해하는지 확인할 수 있습니다.

장점: 비용과 개발 시간을 줄이면서 사용성 문제를 발견할 수 있습니다.

한계: 사람이 대신 처리하므로 AI의 실제 성능이나 자동화 가능성까지 검증할 수는 없습니다.

3-2. 기성 AI 도구를 활용한 프로토타입

ChatGPT와 같은 기존 AI 도구에 프로젝트 지침이나 시스템 프롬프트를 설정해 시제품으로 사용하는 방법입니다.

예를 들어 강의 필기 복습 도우미라면 다음과 같이 설정합니다.

- 사용자가 강의 필기를 붙여 넣습니다.
- AI가 핵심 개념과 퀴즈를 만듭니다.
- 필기에 없는 내용을 임의로 추가하지 않습니다.
- 사용자가 `정답`이라고 입력할 때만 정답과 해설을 보여 줍니다.

이 방법은 사용성뿐 아니라 실제 AI가 만들어 내는 결과의 품질도 확인할 수 있습니다.

구분	오즈의 마법사 테스트	기성 AI 도구 프로토타입
응답 담당	사람이 직접 작성	AI가 생성
주된 검증 대상	채널, 사용 흐름, 출력 형식	사용 흐름과 AI 결과 품질
AI 모델 필요	없어도 됨	필요
대표적인 한계	AI 자동화 성능을 알기 어려움	도구 자체의 기능 한계가 있음

4. 테스트는 어떻게 진행할까요?

강의에서는 두 사람이 역할을 나누어 각각 15분씩 테스트하도록 제안합니다.

1. 역할을 나눕니다. 한 명은 테스트 참여자, 다른 한 명은 진행자가 됩니다.
2. 상황만 알려 줍니다. 예를 들어 “강의가 끝났고, 오늘 배운 내용을 복습하고 싶어요”라고 안내합니다. 사용법까지 설명하지 않습니다.
3. 실제처럼 사용하게 합니다. 참여자는 필기를 붙여 넣거나 사진을 올리는 등 평소처럼 행동합니다.
4. 행동을 관찰합니다. 진행자는 참여자가 멈추는 지점, 결과를 수정하는 부분, 결과를 받은 뒤의 행동을 기록합니다.
5. 관찰을 설계 변경으로 연결합니다. 문제가 반복되면 지침이나 채널을 수정하고, 해결되지 않는 요구는 별도로 기록합니다.

테스트 노트의 핵심 항목

관찰 항목	확인할 질문
처음 한 행동	설명 없이 무엇부터 했나요?
멈춘 지점	어디에서 막히거나 질문했나요?
결과를 받은 직후	공유했나요, 수정했나요, 무시했나요?
수정한 부분	결과에서 무엇을 고쳤나요?

관찰 항목	확인할 질문
요청 사항	사용자가 추가로 원한 것은 무엇인가요?

여기서 중요한 원칙은 사용자가 말한 평가보다 실제 행동을 우선해서 보는 것입니다.

사용자가 “좋네요”라고 말하면서 결과의 절반을 삭제하고 다시 작성한다면, 실제로는 출력 형식이 사용자의 작업에 맞지 않을 가능성이 큽니다.

5. 테스트 결과를 어떻게 설계에 반영할까요?

강의의 복습봇 사례를 보면 세 가지 문제가 발견되었습니다.

문제 1. 퀴즈의 정답이 바로 보임

원인: 출력 형식이 학습자의 문제 풀이 흐름을 고려하지 않았습니다.

→ 정답과 해설은 사용자가 `정답`이라고 요청할 때만 제공하도록 수정합니다.

문제 2. 다음 날 결과를 다시 찾기 어려움

원인: 단체 대화방은 결과를 장기간 보관하고 다시 찾는 용도에 적합하지 않을 수 있습니다.

→ 대화 기록이 기능별로 남는 기성 AI 도구 프로젝트로 채널을 변경합니다.

문제 3. 틀린 퀴즈만 저장하고 싶음

원인: 지침을 바꾸는 것만으로 해결하기 어려운 기능 요구입니다.

→ 파일럿에서 반복되는지 확인하고, 필요하면 별도 저장 기능을 지원하는 웹앱으로 전환합니다.

여기서 기억해야 할 구분이 있습니다.

- 지침으로 해결 가능한 문제: 정답 노출 시점, 출력 순서, 근거 표시, 첫 안내 문구
- 채널을 바꿔야 할 수 있는 문제: 결과 보관, 검색, 사용자별 데이터 관리
- 추가 개발이 필요할 수 있는 문제: 틀린 문제 저장, 학습 이력 관리, 반복 학습 기능

모든 문제를 프롬프트 수정으로 해결할 수 있는 것은 아닙니다. 이것을 구분하는 것이 서비스 기획자의 중요한 역할입니다.

6. 코딩 에이전트에게 무엇을 요청할까요?

강의에서 제시한 프롬프트의 목적은 코드를 작성하기 전에 테스트용 시스템 프롬프트와 첫 안내 메시지를 준비하는 것입니다.

아래 프롬프트는 다른 AI 기능을 테스트할 때도 재사용할 수 있습니다.

코드 없이 AI 기능을 검증하기 위한 프로토타입 프롬프트

다음 기능을 기존 AI 도구의 프로젝트로 먼저 테스트하려고 합니다. 아직 코드를 작성하지 마세요.

- 기능: [기능을 한 문장으로 작성]
- 대상 사용자: [사용 맥락 분석에서 정한 사용자]
- 제공 채널: [채팅, 기성 AI 도구, 웹앱 등]
- 출력 형식: [항목, 순서, 길이, 값이 없을 때의 표기]
- 오류 대응 규칙: [근거 표시, 추측 금지, 대체 안내 등]

다음 결과물을 작성해 주세요.

1. 프로젝트에 붙여 넣을 시스템 프롬프트
2. 처음 사용하는 사람에게 보여 줄 첫 안내 메시지
3. 테스트할 대표 상황 3개
4. 각 상황에서 관찰할 행동과 성공 기준
5. 테스트 결과에 따라 수정할 항목과 수정이 어려운 채널 한계

규칙:

- 사용자가 제공하지 않은 정보는 임의로 만들어 내지 않습니다.
- 출력 형식과 오류 대응 규칙을 구체적인 지침으로 작성합니다.
- 지침으로 해결할 문제와 추가 기능 개발이 필요한 문제를 구분합니다.
- 실제 테스트에서 확인되지 않은 사항을 검증된 사실처럼 표현하지 않습니다.

이 프롬프트를 활용하면 단순히 “AI가 잘 작동하는가?”를 확인하는 데 그치지 않고, 사용자가 실제 업무에서 쓸 수 있는 서비스인지 검증할 수 있습니다.

7. AI 개발자와 1인 AI 사업가에게 왜 중요할까요?

개발자는 기술적으로 가능한 기능을 만들 수 있지만, 사업에서는 사용자가 실제로 필요로 하고 지속적으로 사용하는 기능을 만들어야 합니다.

특히 비개발자 1인 창업가에게 코드 없는 검증은 다음과 같은 의미가 있습니다.

- 개발비를 쓰기 전에 고객의 요구를 확인할 수 있습니다.
- 프롬프트만으로 해결할 수 있는 문제와 별도 개발이 필요한 문제를 구분할 수 있습니다.

- 사용자가 실제로 원하는 기능을 확인해 불필요한 개발을 줄일 수 있습니다.
- 사용자 관찰 결과를 다음 버전의 요구사항(PRD)에 반영할 수 있습니다.

예를 들어 AI Story Partner(ASP)에서 고객의 업무 경험을 바탕으로 콘텐츠 초안을 만드는 기능을 기획한다고 가정하겠습니다.

처음부터 웹앱을 개발하기보다 기존 AI 프로젝트에서 고객에게 경험과 사례를 입력하게 하고, 결과물을 검토하게 할 수 있습니다. 고객이 결과를 어디에서 수정하고 어떤 부분을 다시 작성하는지 관찰하면, 어떤 기능에 투자해야 할지 더 명확해집니다.

이것이 개발 중심의 사고에서 고객 문제 중심의 사고로 전환하는 과정입니다.

8. 이해 점검 퀴즈

Q1. 오즈의 마법사 테스트의 가장 큰 목적은 무엇인가요?

AI 모델의 정확도를 최종 평가한다.

코드 작성 전에 사용자의 행동과 사용 흐름을 검증한다.

서비스의 서버 성능을 측정한다.

Q2. 사용자가 결과를 다음 날 다시 찾을 수 없다고 합니다. 가장 먼저 검토할 것은 무엇인가요?

AI 모델의 이름을 바꾼다.

모든 결과의 글자 수를 줄인다.

현재 채널의 보관·검색 기능과 다른 채널로의 전환 필요성을 확인한다.

Q3. 테스트 참여자가 “좋네요”라고 말했지만 결과의 절반을 지우고 다시 작성했습니다. 어떻게 해석해야 할까요?

칭찬했으므로 출력 형식은 완벽하다.

실제로 수정한 부분을 확인해 출력 형식이나 내용의 문제를 찾아야 한다.

테스트를 중단하고 바로 개발을 시작한다.

정답 확인

9. 반드시 기억할 내용

1. 코드 작성 전에 검증합니다. 잘못된 가정으로 개발하는 시간과 비용을 줄이기 위해서입니다.
2. 오즈의 마법사 테스트는 사람이 시스템 역할을 대신합니다. AI 없이도 채널과 사용 흐름을 시험할 수 있습니다.

3. 기성 AI 도구 프로토타입은 실제 AI 결과까지 검증합니다. 시스템 프롬프트와 출력 규칙을 설정해 사용해 봅니다.
4. 사용자의 말보다 행동을 먼저 봅니다. 멈춘 지점, 수정한 부분, 결과를 공유했는지가 중요한 증거입니다.
5. 문제의 원인에 맞게 수정합니다. 프롬프트로 해결할 문제인지, 채널을 바꿔야 하는지, 별도 기능 개발이 필요한지 구분합니다.
6. 한두 번의 테스트를 모든 고객의 의견으로 일반화하지 않습니다. 여러 사용자에게서 반복되는 문제인지 파일럿을 통해 확인합니다.

10. 한 문장 요약

AI 서비스를 잘 만드는 첫 단계는 코드를 작성하는 것이 아니라, 코드 없이 사용자를 관찰하고 가정을 검증하는 것입니다.

Chapter05

/강의정리 챕터 05. 아키텍처를 정하지 않으면

1. 강의의 목적

이번 강의의 핵심은 웹앱을 개발하기 전에 시스템의 구조와 데이터 처리 방식을 설계해야 한다는 것입니다.

앞선 강의에서는 코드 없이 사용자를 관찰하고, AI 기능이 실제로 필요한지 검증하는 방법을 배웠습니다. 이제 웹앱으로 개발하기로 했다면 한 단계 더 나아가야 합니다.

코딩 에이전트에게 “이런 앱을 만들어 주세요”라고 요청하면 작동하는 화면은 빠르게 만들 수 있습니다. 하지만 요청하지 않은 부분까지 안전하고 확장성 있게 설계해 주리라는 보장은 없습니다.

예를 들어 다음과 같은 문제가 생길 수 있습니다.

- API 키를 앱 코드에 넣어 외부에 노출합니다.
- 데이터를 임시 저장해 서버를 재시작하면 사라집니다.
- 다른 사용자가 내 데이터를 조회하거나 수정할 수 있습니다.
- AI API가 실패했을 때 화면이 멈춥니다.
- 코드를 한 파일에 몰아넣어 기능을 수정하기 어려워집니다.

따라서 코드를 작성하기 전에 다음 세 가지 문서를 준비해야 합니다.

1. 아키텍처 다이어그램: 시스템을 어떤 구성 요소로 만들 것인가?
2. 시퀀스 다이어그램: 사용자가 기능을 실행하면 어떤 순서로 처리되는가?
3. 데이터 모델: 어떤 데이터를 누가 소유하고 어디에 얼마나 저장할 것인가?

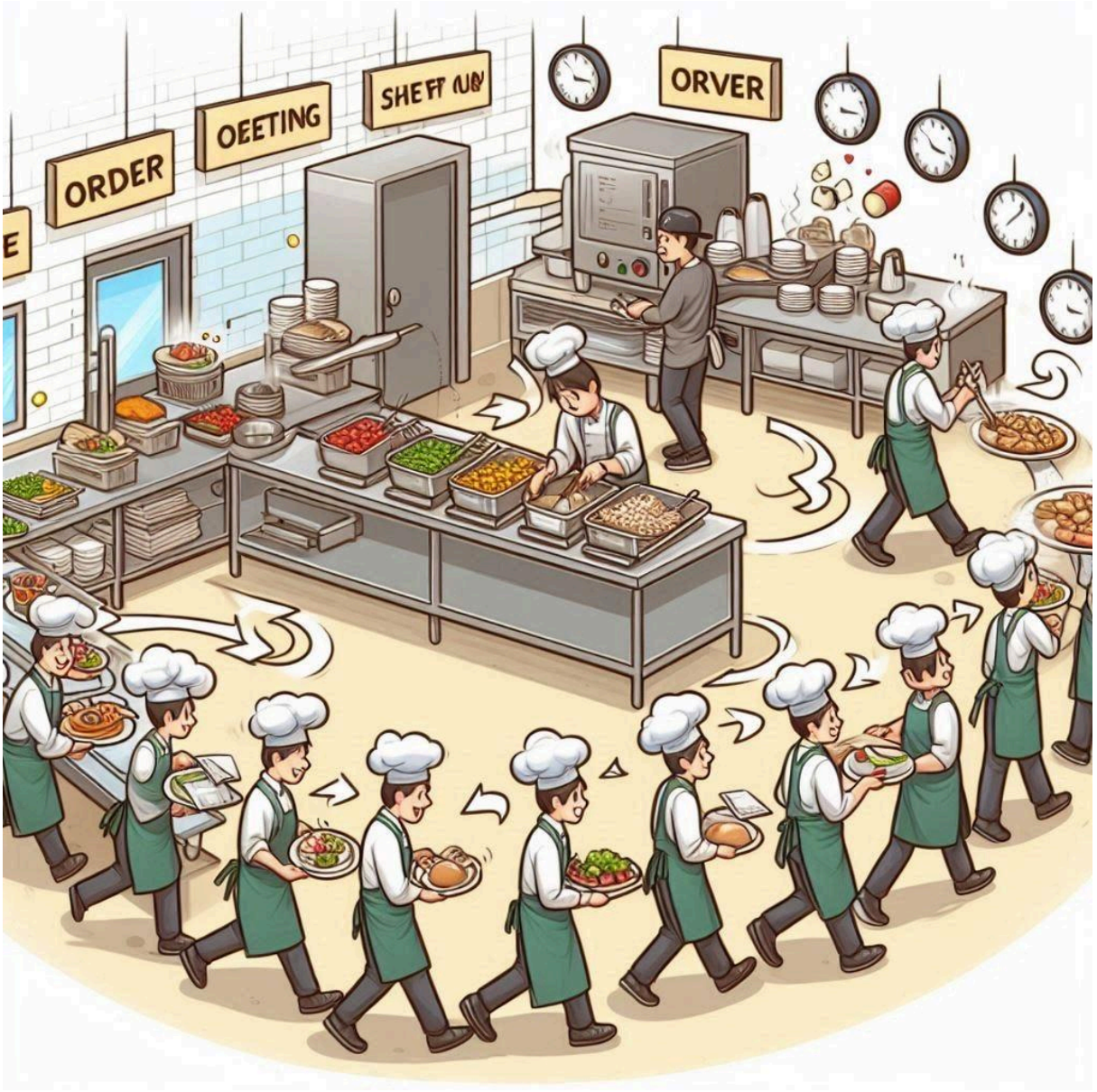
2. 중학생도 이해할 수 있는 설명

AI 웹앱을 하나의 음식점이라고 생각해 보겠습니다.



① 아키텍처 다이어그램 = 식당의 배치도

손님, 주문대, 주방, 식재료 창고, 배달 담당자가 어떻게 연결되어 있는지 보여 줍니다.



② 시퀀스 다이어그램 = 주문 처리 순서

손님이 주문하고, 주방이 음식을 만들고, 직원이 전달하는 과정을 순서대로 보여 줍니다.

③ 데이터 모델 = 식재료 관리대장

어떤 물건을 누가 관리하고, 어디에 보관하며, 언제까지 보관할지 정합니다.

세 문서는 서로 다른 질문에 답합니다.

- 아키텍처: 무엇이 연결되어 있는가?
- 시퀀스: 어떤 순서로 움직이는가?
- 데이터 모델: 무엇을 저장하고 누가 관리하는가?

세 가지를 함께 살펴야 화면이 작동하는 것뿐 아니라 실제 서비스로 운영할 수 있는지도 판단할 수 있습니다.

3. 핵심 개념 ① 아키텍처 다이어그램

아키텍처 다이어그램은 시스템을 구성하는 요소와 요소 사이의 연결 관계를 보여 주는 그림입니다.

강의에서는 사진을 올리면 AI가 중고거래 판매글 초안을 만들어 주는 앱을 예로 들었습니다.

클라이언트

휴대폰 앱 · 사진 업로드 · 결과 표시

백엔드 서버

판매글 API

listings

AI 초안 생성

ai_draft

시세 조회

price

알림 작업 등록

notifications

백엔드에서 외부 서비스와 연결

LLM API

AI 초안 생성

데이터베이스

판매글·사용자 정보

오브젝트 스토리지

사진 파일

작업 큐·워커

나중에 처리할 알림

이해를 돕기 위한 개념도이며, 실제 서비스에서는 구성 요소와 연결이 달라질 수 있습니다.

각 구성 요소는 무슨 일을 할까요?

구성 요소	역할	쉬운 비유
클라이언트	사용자가 보고 조작하는 화면	주문대
백엔드	요청을 처리하고 권한과 업무 규칙을 확인	주방 관리자
LLM API	입력 자료를 바탕으로 AI 결과 생성	전문 요리사
데이터베이스	판매글, 사용자 정보 등 구조화된 데이터 저장	관리대장
오브젝트 스토리지	사진처럼 파일 형태의 데이터 저장	창고
작업 큐·워커	나중에 실행할 작업을 접수하고 처리	대기표와 담당 직원

여기서 가장 먼저 확인할 것은 API 키와 같은 비밀정보가 어디에 있는가입니다.

LLM API 키는 백엔드에 보관하고, 클라이언트는 백엔드에 요청하도록 설계해야 합니다. 클라이언트 코드는 사용자 기기에 전달되므로 그 안에 키를 넣으면 추출될 수 있습니다.

또한 백엔드의 코드를 책임별 모듈로 분리하면 AI 초안 생성 기능을 수정할 때 판매글 등록 기능까지 함께 고장 날 위험을 줄일 수 있습니다.

4. 핵심 개념 ② 시퀀스 다이어그램

아키텍처가 구성 요소의 배치도라면, 시퀀스 다이어그램은 특정 기능이 실행될 때 구성 요소들이 시간 순서대로 주고받는 요청과 응답을 보여 줍니다.

중고거래 앱에서 AI 판매글을 만드는 과정을 살펴보겠습니다.

AI 초안 생성부터 판매글 등록까지

1. 사용자가 사진을 올리고 'AI 초안 생성'을 누릅니다.
2. 클라이언트가 백엔드에 요청합니다. 요청 중에는 버튼을 비활성화해 중복 호출을 줄입니다.
3. 백엔드는 사용 권한과 입력 자료를 확인한 뒤 LLM API를 호출합니다.
4. LLM이 초안을 반환하면 백엔드는 결과를 확인하고 클라이언트에 전달합니다.
5. 사용자가 초안을 확인하고 판매글 등록을 요청하면, 백엔드는 소유자와 입력값 등을 검증한 후 데이터베이스에 저장합니다.
6. 주변 사용자에게 알림을 보내야 한다면 알림 작업을 큐에 등록하고, 워커가 별도로 처리합니다.

이 흐름을 설계하면 화면만 봐서는 알기 어려운 문제도 발견할 수 있습니다.

4-1. LLM 호출 실패에 대비하기

LLM 호출은 시간이 걸리고 비용도 발생합니다. 따라서 다음 사항을 미리 정해야 합니다.

- 타임아웃: 일정 시간 안에 응답하지 않으면 호출을 종료합니다.
- 폴백: 실패했을 때 재시도 안내나 직접 작성 기능 등 대체 경로를 제공합니다.
- 중복 호출 방지: 처리 중에는 버튼을 비활성화하거나 중복 요청을 식별합니다.
- 데이터 검증: AI 결과가 비어 있거나 잘못된 형식이면 정상 결과처럼 저장하지 않습니다.

4-2. 금지 품목은 AI 호출 전에 걸러내기

예를 들어 판매 금지 품목에 해당하는 입력이라면, 서버가 이를 확인한 뒤 적절한 오류 응답을 보내고 이후의 AI 호출을 진행하지 않도록 설계할 수 있습니다.

강의에서는 이 경우 `422` 응답을 예로 들었습니다. 이는 요청의 내용이 처리 규칙에 맞지 않는 상황을 나타낼 때 사용할 수 있는 HTTP 상태 코드입니다.

4-3. 오래 걸리는 작업은 비동기로 처리하기

판매글을 등록하면서 주변 사용자 수백 명에게 알림을 보낸다고 가정해 보겠습니다.

등록 요청 안에서 알림 발송까지 모두 기다리면 사용자는 오래 기다려야 합니다.

대신 다음과 같이 처리합니다.

1. 판매글을 등록합니다.
2. 알림 작업을 큐에 넣습니다.
3. 등록이 완료되면 `201 Created` 응답을 보냅니다.
4. 워커가 큐에서 작업을 꺼내 알림을 전송합니다.

즉, 사용자에게 즉시 알려 줄 수 있는 결과와 뒤에서 처리해도 되는 작업을 분리하는 것입니다.

5. 핵심 개념 ③ 데이터 모델

데이터 모델은 시스템이 관리하는 데이터의 종류와 속성을 정리한 문서입니다.

강의에서는 특히 네 가지 항목을 확인합니다.

- 생성 시점
- 소유자
- 저장소
- 보존 기간

중고거래 앱의 개념적 데이터 모델 예시

데이터	생성 시점	소유자	저장 위치	보존 기준
판매글	사용자가 등록할 때	판매자	데이터베이스	서비스 정책에 따름
사진	사용자가 업로드할 때	관련 사용자·서비스 정책에 따라 결정	오브젝트 스토리지	목적과 정책에 따름
AI 초안	AI 생성 완료 시	사용자 작업에 연결	미저장 또는 임시 저장	설계에서 결정
알림 작업	알림 요청 시	관련 사용자·작업	작업 큐	처리 완료 및 재시도 정책에 따름

이 표는 설계 예시입니다. 실제 소유권, 저장 위치와 보존 기간은 서비스의 개인정보 처리 방침과 업무 요구사항에 맞게 확정해야 합니다.

5-1. 소유자는 보안과 연결됩니다

판매글의 소유자가 A라면 B가 그 판매글을 수정하거나 삭제하지 못하도록 해야 합니다.

따라서 백엔드는 판매글 수정·삭제 요청이 올 때마다 요청한 사용자가 실제 소유자인지 확인해야 합니다. 단순히 화면에서 수정 버튼을 숨기는 것만으로는 충분하지 않습니다.

5-2. 보존 기간은 개인정보와 비용에 연결됩니다

사용자가 탈퇴했는데 사진이 스토리지에 계속 남아 있다면 개인정보 관리 문제가 발생할 수 있고, 저장 비용도 지속적으로 발생할 수 있습니다.

또한 거래 내역처럼 여러 당사자와 관련된 데이터는 한 사람의 삭제 요청만으로 바로 삭제해도 되는지 별도로 검토해야 합니다.

데이터를 저장할지뿐 아니라 누가 소유하고, 언제 삭제할지도 설계 대상입니다.

6. 세 가지 문서를 함께 사용해야 하는 이유

아키텍처 다이어그램

구성 요소와 연결 관계를 확인합니다.

API 키가 안전한 위치에 있는지, 불필요하게 한 모듈에 기능이 몰려 있지는 않은지 살펴봅니다.

시퀀스 다이어그램

요청의 처리 순서를 확인합니다.

실패 시 대체 경로, 중복 요청, 비동기 작업이 적절한지 살펴봅니다.

데이터 모델

저장되는 데이터와 관리 책임을 확인합니다.

사용자별 접근 권한, 저장 기간, 삭제 정책을 검토합니다.

세 문서는 별개의 산출물이지만 서로 연결됩니다. 아키텍처에서 데이터베이스를 사용하기로 했다면 데이터 모델에 저장 대상과 소유자를 정해야 합니다. 시퀀스에서는 그 데이터가 언제 저장되고 실패하면 어떻게 되는지 정해야 합니다.

7. 코딩 에이전트에게 개발을 요청하기 전에

웹앱을 만들 때는 단순히 화면과 기능만 설명하지 말고, 다음 항목을 함께 요청하는 것이 좋습니다.

웹앱 개발 전 아키텍처 설계 요청

아직 코드를 작성하지 말고, 다음 기능을 구현하기 위한 설계 문서를 먼저 작성해 주세요.

1. 서비스 개요

- 대상 사용자: [사용자]
- 핵심 기능: [기능]
- 제공 채널: [웹앱 또는 모바일 앱]

2. 아키텍처 다이어그램

- 클라이언트, 백엔드, 데이터베이스, 파일 저장소, 외부 API를 구분합니다.
- 백엔드 모듈별 책임과 호출 관계를 표시합니다.
- API 키 등 비밀정보가 클라이언트에 노출되지 않도록 설계합니다.

3. 시퀀스 다이어그램

- 정상 처리 흐름을 작성합니다.
- 타임아웃, 외부 API 실패, 중복 요청, 잘못된 입력의 처리 흐름을 포함합니다.
- 오래 걸리는 작업은 동기 처리와 비동기 처리 중 적절한 방식을 선택하고 이유를 설명합니다.

4. 데이터 모델

각 데이터에 대해 다음 항목을 정리합니다.

- 생성 시점
- 소유자
- 저장 위치
- 보존 기간과 삭제 조건

5. 보안과 운영

- 사용자별 조회·수정·삭제 권한 검증
- API 키와 비밀정보 관리
- 오류 응답과 로그 처리
- 비용과 중복 호출 관리
- 데이터 백업 및 복구 필요성

6. 최종 검토

- 설계상 가정과 아직 결정되지 않은 사항을 구분합니다.
- 각 결정의 이유와 대안을 설명합니다.
- 초보 개발자도 이해할 수 있도록 용어를 설명합니다.
- 설계 검토가 끝나기 전에는 코드를 작성하지 않습니다.

이렇게 요청하면 코딩 에이전트가 임의로 선택한 구조를 나중에 뒤늦게 발견하는 위험을 줄일 수 있습니다. 다만 설계 문서를 받았다고 끝나는 것은 아닙니다. 특히 인증·권한·비밀정보 관리처럼 보안에 영향을 주는 항목은 실제 구현에서도 확인해야 합니다.

8. AI 개발자와 1인 창업가에게 왜 중요할까요?

비개발자가 코딩 에이전트를 활용하면 개발 속도를 크게 높일 수 있습니다. 하지만 빠르게 만든 앱이 곧바로 안전하고 운영 가능한 앱이라는 뜻은 아닙니다.

아키텍처를 이해하면 다음과 같은 역량을 갖추게 됩니다.

- 개발자나 코딩 에이전트가 제안한 구조를 검토할 수 있습니다.
- 오류가 발생했을 때 어느 구성 요소에서 문제가 생겼는지 추적할 수 있습니다.
- 사용자 증가에 따라 어떤 부분을 개선해야 하는지 판단할 수 있습니다.
- 기능 개발 전에 데이터 소유권과 보안 위험을 발견할 수 있습니다.
- 외주 개발이나 협업 과정에서 요구사항을 더 명확하게 전달할 수 있습니다.

특히 ASP와 같은 1인 AI 사업에서는 모든 코드를 직접 작성하는 것보다 무엇을 만들어야 하는지, 어떻게 연결되어야 하는지, 어떤 위험을 막아야 하는지 판단하는 능력이 중요합니다.

9. 이해 점검 퀴즈

Q1. 주변 사용자에게 알림을 보내는 작업을 등록 요청과 분리해 작업 큐에 넣는 가장 중요한 이유는 무엇인가요?

알림 데이터는 저장할 필요가 없기 때문입니다.

알림 대상이 많아도 등록 요청은 빠르게 응답하고, 알림은 별도로 처리하기 위해서입니다.

알림 문구를 반드시 LLM이 생성해야 하기 때문입니다.

알림 기능을 첫 버전에서 제외하기 위해서입니다.

정답 확인

Q2. LLM API가 한 시간 동안 장애를 겪었다고 가정해 보세요. 중고거래 앱에서 영향을 받는 기능과 영향을 받지 않을 수 있는 기능을 나눠 적어 보세요.

주관식 답변 검토받기

주관식 예시 답안

- 영향을 받는 기능: AI 판매글 초안 생성처럼 LLM API를 직접 호출하는 기능입니다. API 장애에 대비한 폴백이 없다면 사용자는 초안을 받지 못할 수 있습니다.
- 영향을 받지 않을 수 있는 기능: 기존 판매글 조회, 이미 작성된 판매글 수정·삭제, 데이터베이스에 직접 저장하는 기능 등은 LLM에 의존하지 않도록 설계했다면 계속 작동할 수 있습니다.
- 주의할 점: 실제 영향 범위는 시스템 설계에 따라 다릅니다. 예를 들어 판매글 등록 과정에서 LLM 호출을 필수로 요구하면 등록 기능도 영향을 받습니다. 기존 판매글 조회가 정상 작동하려면 데이터베이스와 백엔드도 정상이어야 합니다.

10. 반드시 기억할 내용

1. 아키텍처 다이어그램은 구성 요소와 연결 관계를 보여 줍니다.
2. 시퀀스 다이어그램은 요청부터 응답까지의 처리 순서와 실패 흐름을 보여 줍니다.
3. 데이터 모델은 저장 대상, 소유자, 저장 위치, 보존 기간을 정합니다.
4. API 키는 클라이언트가 아니라 백엔드에서 안전하게 관리해야 합니다.
5. 오래 걸리는 작업은 작업 큐와 워커를 활용해 비동기로 분리할 수 있습니다.
6. 데이터를 저장하는 것만큼 누가 접근할 수 있고 언제 삭제할지 정하는 것도 중요합니다.

7. 코딩 에이전트에게 코드를 요청하기 전에 설계 문서를 검토하면 나중에 발생할 구조적 문제를 줄일 수 있습니다.

한 문장 요약

코딩 에이전트에게 무엇을 만들지 알려 주는 것만으로는 부족하며, 시스템이 어떻게 연결되고 데이터를 어떻게 처리할지까지 설계해야 안전하고 유지보수하기 쉬운 AI 서비스를 만들 수 있습니다.

Chapter06

/강의정리 챕터 06. 시퀀스 다이어그램부터

1. 강의의 목적

이번 강의의 핵심은 내가 만들 AI 서비스를 직접 설계하고, 코딩 에이전트에게 개발을 맡기기 전에 설계의 빈틈을 확인하는 것입니다.

챕터 05에서는 웹앱을 설계할 때 필요한 세 가지 문서를 배웠습니다.

1. 시퀀스 다이어그램: 기능이 어떤 순서로 처리되는가?
2. 아키텍처 다이어그램: 어떤 구성 요소가 필요한가?
3. 데이터 모델: 어떤 데이터를 저장하고 누가 소유하는가?

이번에는 이 문서들을 실제로 만들 서비스에 적용합니다.

중요한 차이는 이번에는 아키텍처부터 그리지 않고, 시퀀스 다이어그램부터 그린다는 점입니다. 사용자의 행동 하나에서 시작하면 실제로 필요한 구성 요소와 데이터가 자연스럽게 드러나기 때문입니다.

2. 중학생도 이해할 수 있는 설명

AI 서비스를 음식점에 비유해 보겠습니다.

처음부터 식당의 모든 공간과 장비를 설계하면 정작 손님이 어떤 순서로 주문하고 음식을 받는지 놓칠 수 있습니다.

따라서 먼저 손님 한 명의 주문 과정을 그려 봅니다.

1

시퀀스 다이어그램부터 작성

손님이 주문하고, 주방이 조리하고, 직원이 음식을 전달하는 순서를 그립니다.

2

아키텍처 다이어그램 작성

주문대, 주방, 창고, 직원 등 실제로 필요한 구성 요소를 정리합니다.

3

데이터 모델 작성

주문서와 재고를 어디에 보관하고 누가 관리할지 정합니다.

AI 서비스도 마찬가지입니다. 사용자가 버튼을 누른 뒤 어떤 일이 일어나는지부터 그리면, 필요한 서버와 저장소, 외부 AI 호출, 오류 대응을 구체적으로 정할 수 있습니다.

3. 직접 설계하는 순서

1단계. 시퀀스 다이어그램

가장 중요한 사용자 동작을 하나 선택합니다.

예를 들어 강의 복습 노트 서비스라면 다음과 같습니다.

> 사용자가 강의 필기를 붙여 넣고 퀴즈 만들기를 요청한다.

이 동작을 중심으로 구성 요소를 나열하고 요청과 응답을 시간순으로 연결합니다.

강의 복습 노트 서비스의 기본 흐름

1. 사용자가 필기를 입력하고 퀴즈 생성을 요청합니다.
2. 클라이언트가 백엔드에 요청을 전달합니다.
3. 백엔드가 입력을 검증하고 LLM API를 호출합니다.
4. LLM이 핵심 개념과 퀴즈를 생성합니다.
5. 백엔드가 결과를 검증한 뒤 클라이언트에 반환합니다.
6. 사용자는 결과를 확인하고, 설계에 따라 필요한 데이터만 저장합니다.

개념을 이해하기 위한 예시입니다. 실제 서비스의 저장 방식과 구성 요소는 직접 결정해야 합니다.

여기에 앞선 강의에서 배운 처리 방식과 오류 대응을 추가합니다.

- LLM 응답이 늦으면 어떻게 할까요?

- LLM 호출이 실패하면 어떤 안내를 보여 줄까요?
- 필기가 비어 있거나 읽을 수 없다면 어떻게 할까요?
- 사용자가 버튼을 여러 번 누르면 어떻게 할까요?
- 결과를 저장하지 않기로 했다면 어디까지 보관할까요?

조건에 따라 흐름이 달라지는 부분은 시퀀스 다이어그램의 `alt` 영역으로 표현할 수 있습니다. `alt`는 쉽게 말해 **"이 조건이면 A, 다른 조건이면 B"**라는 분기입니다.

2단계. 아키텍처 다이어그램

시퀀스 다이어그램에 등장한 구성 요소를 모아 시스템 전체 구조를 그립니다.

예를 들어 다음 구성 요소가 필요할 수 있습니다.

- 클라이언트: 사용자가 필기를 입력하고 결과를 보는 화면
- 백엔드: 입력 검증, AI 호출, 결과 검증
- LLM API: 복습 자료와 퀴즈 생성
- 데이터베이스: 사용자 정보나 학습 이력을 저장하는 경우
- 작업 큐와 워커: 시간이 오래 걸리는 작업을 별도로 처리하는 경우

여기서 모든 서비스에 데이터베이스나 작업 큐가 반드시 필요한 것은 아닙니다. 실제 요구사항에 필요한 구성 요소만 선택하는 것이 중요합니다.

백엔드도 책임에 따라 모듈로 분리합니다. 예를 들어 `quiz`는 퀴즈 생성 관련 처리, `study_notes`는 복습 노트 관련 처리를 담당하도록 나눌 수 있습니다.

그리고 LLM API 키 등 비밀정보는 백엔드에서 관리하도록 명시합니다.

3단계. 데이터 모델

이번에는 시퀀스 다이어그램에서 데이터베이스나 파일 저장소로 향하는 화살표를 찾아봅니다.

각 화살표가 저장하는 데이터를 데이터 모델에 기록합니다.

데이터	생성 시점	소유자	저장 위치	보존 기간
강의 필기 원문	사용자가 입력할 때	사용자	저장 안 함 또는 별도 저장소	정책에 따라 결정
AI 복습 결과	생성 완료 시	사용자 작업에 연결	저장 안 함 또는 데이터베이스	정책에 따라 결정
퀴즈 풀이 기록	사용자가 답을 제출할 때	해당 사용자	데이터베이스	학습 이력 정책에 따라 결정

데이터	생성 시점	소유자	저장 위치	보존 기간
오류 로그	처리 실패 시	서비스 운영 주체	로그 저장소	운영·보안 정책에 따라 결정

위 표는 예시이며, 모든 항목을 반드시 저장해야 한다는 의미는 아닙니다.

특히 화면에 표시된다고 해서 반드시 저장해야 하는 것은 아닙니다. 반대로 화면에 보이지 않더라도 오류 로그나 비동기 작업처럼 내부적으로 저장해야 하는 데이터가 있을 수 있습니다.

데이터마다 소유자와 보존 기간을 정해야 사용자 권한, 개인정보 보호, 저장 비용 문제도 함께 검토할 수 있습니다.

4. 설계 점검: 다섯 가지 항목

세 문서를 작성한 뒤에는 다음 항목이 모두 정해졌는지 확인합니다.

아키텍처 점검표

0/5 확인

- [ ] API 키 위치
  LLM API 키와 스토리지 접근 키가 클라이언트에 노출되지 않도록 했나요?
- [ ] 데이터 저장소
  필기, 결과, 학습 기록 등을 어디에 저장할지 또는 저장하지 않을지 정했나요?
- [ ] 데이터 소유자
  사용자별 데이터의 소유자와 조회·수정·삭제 권한을 정했나요?
- [ ] 외부 API 실패 처리
  타임아웃, 오류 안내, 재시도 또는 대체 흐름을 정했나요?
- [ ] 모듈 분리
  기능별 책임을 나눠 수정과 유지보수가 쉽도록 했나요?

정하지 않은 항목을 그대로 두면 코딩 에이전트가 대신 결정하게 됩니다. 그 결정이 처음에는 잘 작동하더라도 사용자가 늘거나 기능을 수정할 때 문제가 드러날 수 있습니다.

5. 코딩 에이전트에게 설계 리뷰 받기

설계를 완성했다면 에이전트에게 검토를 요청할 수 있습니다. 여기서 중요한 점은 에이전트가 설계를 임의로 바꾸게 하지 않는 것입니다.

AI 서비스 설계 리뷰 요청

아래는 제가 설계한 서비스의 시퀀스 다이어그램, 아키텍처 다이어그램, 데이터 모델입니다.

설계를 직접 수정하거나 대신 결정하지 마세요.

빠졌거나 아직 결정되지 않은 사항을 질문 형태로 최대 5개까지 알려 주세요.

특히 다음 항목을 검토해 주세요.

1. API 키와 비밀정보의 위치
2. 데이터 저장소와 데이터 소유자
3. 외부 API 실패 시 처리 흐름
4. 구성 요소 사이의 호출 관계
5. 모듈 분리와 데이터 보존 정책

질문마다 왜 확인이 필요한지 한 문장으로 설명해 주세요.

제가 검토한 뒤 직접 설계를 수정하겠습니다.

[여기에 세 문서의 내용 또는 이미지 첨부]

에이전트가 질문했다고 해서 모두 반영할 필요는 없습니다. 질문을 검토한 뒤 동의하는 내용은 설계에 반영하고, 동의하지 않는 내용은 이유를 기록합니다.

예를 들어 “사용자별 학습 이력을 저장해야 하지 않나요?”라는 질문을 받았다면, 학습 이력이 첫 버전의 범위에 포함되는지 먼저 판단해야 합니다. MVP에서 저장 기능을 제외하기로 했다면, 그 이유와 향후 추가 조건을 기록하면 됩니다.

6. AI 개발자와 1인 창업가에게 왜 중요할까요?

이번 강의는 단순히 다이어그램을 그리는 기술을 배우는 시간이 아닙니다. AI에게 개발을 맡기더라도 서비스의 중요한 결정을 직접 내리는 연습입니다.

특히 1인 AI 사업에서는 다음과 같은 이점이 있습니다.

- 개발을 시작하기 전에 필요한 구성 요소를 파악할 수 있습니다.
- 기능을 추가할 때 기존 시스템에 미칠 영향을 예상할 수 있습니다.
- 코딩 에이전트가 선택한 구조를 검토하고 질문할 수 있습니다.

- 고객 데이터와 API 비용을 관리하는 기준을 만들 수 있습니다.
- 이후 PRD에 들어갈 아키텍처 요구사항을 구체화할 수 있습니다.

ASP나 My-Story-Maker 같은 서비스를 개발할 때도 동일하게 적용할 수 있습니다. 처음에는 간단한 AI 기능으로 시작하더라도 사용자의 자료를 저장하거나 여러 사람이 함께 사용하는 단계로 넘어가면 데이터 소유권과 접근 권한이 중요해지기 때문입니다.

7. 이해 점검 퀴즈

Q1. 시퀀스 다이어그램을 먼저 작성하는 이유로 가장 적절한 것은 무엇인가요?

세 문서 중 가장 예쁘게 그릴 수 있기 때문입니다.

사용자 동작의 처리 순서를 먼저 확인하면 필요한 구성 요소와 데이터가 드러나기 때문입니다.

시퀀스 다이어그램만 있으면 나머지 문서가 필요 없기 때문입니다.

코딩 에이전트가 설계 결정을 모두 대신할 수 있기 때문입니다.

정답 확인

8. 주관식 퀴즈 예시 답안

질문: 시퀀스 다이어그램을 그리고 나서 아키텍처 다이어그램과 데이터 모델을 채울 때, 시퀀스에서 처음 드러난 구성 요소나 데이터가 있었나요? 그것을 정하지 않은 채 코딩 에이전트에게 개발을 요청했다면 어떻게 됐을까요?

강의의 복습 노트 서비스를 예로 들면 다음과 같이 답할 수 있습니다.

> 시퀀스 다이어그램을 그리면서 LLM API뿐 아니라 오류 로그와 학습 이력의 저장 여부, 외부 API가 실패했을 때의 대체 흐름도 검토할 필요가 있음을 발견했습니다. 이런 사항을 정하지 않은 채 코딩 에이전트에게 개발을 요청했다면, 에이전트가 임시 저장소를 선택하거나 오류 처리를 생략할 수 있습니다. 또한 사용자별 데이터 소유권을 정하지 않으면 다른 사용자가 데이터에 접근하는 문제도 발생할 수 있습니다. 따라서 API 키 위치, 저장소, 데이터 소유자, 외부 API 실패 처리, 모듈 분리를 미리 결정하고 설계 리뷰를 통해 누락된 항목을 확인해야 합니다.

실제 답변에서는 본인이 설계하면서 새롭게 발견한 구성 요소나 데이터를 구체적으로 적는 것이 좋습니다.

9. 반드시 기억할 내용

1. 시퀀스 다이어그램부터 시작합니다. 중요한 사용자 동작 하나를 선택해 처리 순서를 그립니다.

2. 아키텍처 다이어그램은 처리 순서에서 필요한 구성 요소를 추려 작성합니다.
3. 데이터 모델은 저장하는 데이터뿐 아니라 저장하지 않는 데이터도 명시합니다.
4. API 키, 저장소, 소유자, 실패 처리, 모듈 분리는 반드시 확인합니다.
5. 코딩 에이전트는 설계의 빈틈을 찾는 검토자로 활용합니다. 중요한 결정은 직접 내립니다.
6. 완성한 세 문서는 PRD의 아키텍처 항목으로 연결됩니다.

한 문장 요약

내 서비스의 핵심 사용자 동작을 시퀀스 다이어그램으로 먼저 설계하고, 그 흐름에서 아키텍처와 데이터 모델을 도출한 뒤, 코딩 에이전트에게 빠진 부분을 질문하도록 해야 합니다.

Chapter07

/강의정리 챕터 07. 시퀀스 다이어그램부터 설계하기

핵심 주제: AI 서비스 설계 · 시스템 구조 · 데이터 관리 · 코딩 에이전트 활용

1. 강의의 목적

이번 강의의 목적은 AI 서비스의 핵심 동작을 먼저 설계하고, 그 흐름을 바탕으로 시스템 구조와 데이터 모델을 구체화하는 것입니다.

세 가지 설계 문서의 역할은 다음과 같습니다.

- 시퀀스 다이어그램: 사용자의 요청부터 결과까지 어떤 순서로 처리되는가?
- 아키텍처 다이어그램: 어떤 구성 요소가 필요하고, 서로 어떻게 연결되는가?
- 데이터 모델: 어떤 데이터를 저장하며, 누가 소유하고 관리하는가?

핵심은 아키텍처부터 무작정 그리지 않는 것입니다. 사용자의 핵심 행동 하나를 먼저 정하고, 실제 처리 과정을 따라가면 필요한 구성 요소와 데이터가 자연스럽게 드러납니다.

2. 중학생도 이해할 수 있는 설명

음식점을 새로 연다고 생각해 보겠습니다.

1. 시퀀스 다이어그램은 손님이 주문하고 음식을 받기까지의 업무 순서표입니다.

2. 아키텍처 다이어그램은 주문대, 주방, 창고 등 필요한 시설의 배치도입니다.
3. 데이터 모델은 주문서와 재고를 어디에 보관하고 누가 관리할지 정한 관리 장부입니다.

AI 서비스도 같습니다. 사용자가 버튼을 누른 뒤 어떤 일이 일어나는지 먼저 그려야 필요한 서버, AI 모델, 데이터베이스, 오류 처리 방식을 제대로 결정할 수 있습니다.

3. 설계하는 세 단계

1단계. 시퀀스 다이어그램

사용자가 강의 필기를 입력하고 퀴즈 생성을 요청하면, 클라이언트 → 백엔드 → LLM API → 결과 검증 → 사용자 화면 순서로 처리됩니다.

확인할 것: 입력 오류, 응답 지연, API 실패, 중복 요청

2단계. 아키텍처 다이어그램

시퀀스에 등장한 구성 요소를 모아 시스템 구조를 설계합니다. 클라이언트, 백엔드, LLM API, 필요에 따른 데이터베이스나 작업 큐가 여기에 해당합니다.

확인할 것: 구성 요소의 역할, 호출 관계, 기능별 모듈 분리

3단계. 데이터 모델

필기 원문, AI 생성 결과, 퀴즈 풀이 기록, 오류 로그 등을 검토합니다. 각 데이터의 저장 여부와 소유자, 저장 위치, 보존 기간을 결정합니다.

확인할 것: 저장하지 않을 데이터까지 명시하기

중요한 점은 모든 서비스에 데이터베이스나 작업 큐가 반드시 필요한 것은 아니라는 것입니다. 실제 요구사항에 맞춰 필요한 구성 요소만 선택합니다.

4. 설계에서 반드시 점검할 다섯 가지

점검 항목	확인할 질문
API 키	비밀정보가 사용자 화면이나 클라이언트에 노출되지 않나요?
데이터 저장소	무엇을 어디에 저장하고, 무엇은 저장하지 않나요?
데이터 소유자	누가 데이터를 조회·수정·삭제할 수 있나요?
외부 API 실패	시간 초과나 호출 실패 시 어떤 안내와 대체 흐름을 제공하나요?
모듈 분리	기능별 책임이 나뉘어 수정과 유지보수가 쉬운가요?

이 항목들을 정하지 않으면 코딩 에이전트가 임의로 결정할 수 있습니다. 처음에는 작동하더라도 사용자가 늘거나 기능이 바뀌면 보안, 비용, 유지보수 문제가 발생할 수 있습니다.

5. 코딩 에이전트는 설계자가 아니라 검토자로 활용하기

설계 문서를 작성한 뒤에는 코딩 에이전트에게 빠진 부분을 찾아 달라고 요청합니다. 다만 설계를 마음대로 수정하게 하지 말고, 미결정 사항을 질문 형태로 최대 5개까지 제시하도록 하는 것이 핵심입니다.

예를 들어 학습 이력을 저장할지 질문받았다면, 무조건 저장하는 것이 아니라 첫 번째 MVP에 필요한 기능인지 판단합니다. 저장하지 않기로 했다면 그 이유와 향후 도입 조건을 기록하면 됩니다.

이 방식은 AI에게 개발을 맡기면서도 중요한 사업·기술적 결정은 직접 내릴 수 있게 해 줍니다.

6. AI 개발자와 1인 창업가에게 중요한 이유

현재 추진하시는 ASP와 My-Story-Maker 같은 서비스에도 그대로 적용할 수 있습니다.

- 개발을 시작하기 전에 필요한 기능과 구성 요소를 파악할 수 있습니다.
- 고객 데이터의 소유권과 접근 권한을 미리 설계할 수 있습니다.
- 코딩 에이전트가 제안한 구조를 이해하고 검토할 수 있습니다.
- AI API 비용과 오류 대응을 고려해 서비스 운영 위험을 줄일 수 있습니다.
- 설계 문서를 PRD의 아키텍처 요구사항으로 연결할 수 있습니다.

비개발자 창업가에게 필요한 것은 모든 코드를 직접 작성하는 능력만이 아닙니다. 무엇을 만들지 결정하고, AI가 만든 결과가 적절한지 판단할 수 있는 설계 역량도 중요합니다.

7. 이해 점검 퀴즈

질문. 시퀀스 다이어그램을 먼저 작성하는 가장 중요한 이유는 무엇일까요?

세 가지 문서 중 가장 쉽게 그릴 수 있기 때문입니다.

사용자 동작의 처리 순서를 확인하면 필요한 구성 요소와 데이터가 드러나기 때문입니다.

시퀀스 다이어그램만 있으면 나머지 문서가 필요 없기 때문입니다.

코딩 에이전트가 모든 설계 결정을 대신할 수 있기 때문입니다.

정답 확인

8. 반드시 기억할 내용

1. 시퀀스 → 아키텍처 → 데이터 모델 순서로 설계합니다.
2. 핵심 사용자 행동 하나에서 시작해 실제 처리 흐름을 그립니다.

3. 데이터는 저장할 항목뿐 아니라 저장하지 않을 항목도 결정합니다.
4. API 키, 데이터 소유자, 저장소, 실패 처리, 모듈 분리를 확인합니다.
5. 코딩 에이전트는 설계를 대신 결정하는 존재가 아니라 누락을 찾아 주는 검토자로 활용합니다.
6. 최종 설계는 PRD의 아키텍처 요구사항으로 연결합니다.

한 문장 요약

사용자의 핵심 행동을 시퀀스 다이어그램으로 먼저 설계하고, 그 흐름에서 아키텍처와 데이터 모델을 도출한 뒤, 코딩 에이전트의 질문을 통해 설계의 빈틈을 점검합니다.

Chapter08

/강의정리 챕터 08. AI 서비스 설계부터 PRD까지 종합 정리

핵심 주제: 제공 방식 · 사용성 검증 · 아키텍처 · PRD · 서비스 구현

1. 강의의 목적

이번 강의는 지금까지 배운 AI 서비스 설계 과정을 하나로 연결해 정리하는 시간입니다.

핵심은 AI 기능을 만드는 것 자체보다 사용자가 실제로 이용할 수 있는 서비스로 설계하고, 검증하고, 구현할 준비를 하는 것입니다.

전체 과정은 다음과 같습니다.

① 사용 맥락 파악

사용자는 어디에서, 언제 이 기능을 이용하는가?

② 제공 방식과 처리 흐름 설계

어떤 채널에서, 어떤 순서와 방식으로 처리하는가?

③ 코드 없이 사용성 검증

사용자가 실제로 원하는 방식으로 이용하는가?

④ 아키텍처 설계 및 PRD 작성

필요한 구성 요소, 데이터, 비용, 접근 권한을 문서화

⑤ 개발·배포·운영

설계한 요구사항을 실제 서비스로 구현

2. 중학생도 이해할 수 있는 설명

새로운 식당을 연다고 생각해 보겠습니다.

- 제공 방식: 손님이 매장, 전화, 앱 중 어디에서 주문하는 것이 편한지 결정합니다.
- 검증: 실제로 음식을 제공해 보고 손님이 어떻게 행동하는지 관찰합니다.
- 아키텍처: 주방, 주문대, 창고와 각 시설의 연결 관계를 설계합니다.
- PRD: 메뉴, 주문 방식, 비용, 운영 규칙을 하나의 계획서로 정리합니다.
- 개발과 운영: 계획서에 따라 식당을 만들고 실제로 운영합니다.

AI 서비스도 마찬가지입니다. 기술적으로 구현할 수 있다는 사실만으로 성공이 보장되지는 않습니다. 사용자가 편리하게 이용하고, 서비스가 안정적으로 작동하며, 운영할 수 있는 비용 구조까지 갖춰야 합니다.

3. 오늘 배운 네 가지 핵심

① 제공 방식: 기능보다 사용 맥락이 먼저입니다

같은 AI 기능이라도 사용자가 어디에서, 언제, 어떤 목적으로 이용하는지에 따라 제공 채널이 달라집니다.

- 사용자가 이미 사용하는 도구 안에서 해결할 수 있는지 확인합니다.
- 처음부터 별도 앱을 만들기보다 기성 AI 도구로 검증합니다.
- 처리 시간이 짧으면 동기 방식, 오래 걸리면 비동기 방식, 정해진 시간에 실행하면 스케줄 방식을 검토합니다.
- 오류의 영향이 크면 출처 표시, 사람의 승인(HITL), 담당자에게 전달하는 에스컬레이션 등을 추가합니다.

② 검증: 말보다 실제 행동을 관찰합니다

오즈의 마법사 테스트와 기성 AI 도구 프로토타입을 활용하면 코드를 작성하기 전에 사용성을 검증할 수 있습니다.

예를 들어 사용자가 “이 기능이 유용하네요”라고 말하더라도 실제로 결과를 확인하고 다시 이용하는지는 별개의 문제입니다.

따라서 사용자의 의견뿐 아니라 실제 행동을 기준으로 설계를 수정해야 합니다.

③ 아키텍처: 처리 순서에서 시스템 구조를 도출합니다

웹앱을 설계할 때는 다음 세 문서를 연결합니다.

문서	답해야 할 질문
시퀀스 다이어그램	요청부터 결과까지 어떤 순서로 처리되는가?
아키텍처 다이어그램	어떤 시스템 구성 요소가 필요한가?
데이터 모델	어떤 데이터를 저장하고 누가 관리하는가?

코딩 에이전트에게는 설계를 대신 결정하도록 하기보다 빠진 항목을 질문하게 합니다. API 키, 데이터 소유권, 저장 정책, 외부 API 오류 처리 등을 직접 검토하는 것이 중요합니다.

④ PRD: 서비스에 필요한 결정을 하나로 모읍니다

PRD(Product Requirements Document)는 제품 요구사항을 정리한 문서입니다.

기능 목록만 적는 것이 아니라 접근 제어, 데이터 처리, 비용, 파일럿 계획, 아키텍처 등을 함께 기록합니다.

이후 배우는 배포와 운영 기술은 별개의 지식이 아니라, PRD에 기록한 요구사항을 실제로 구현하기 위한 수단입니다.

4. AI 개발자와 1인 창업가에게 중요한 이유

비개발자 창업가에게 이 과정이 중요한 이유는 직접 모든 코드를 작성하지 않더라도 서비스의 방향과 중요한 결정을 주도할 수 있기 때문입니다.

특히 다음 세 가지를 기억하시면 좋겠습니다.

1. 먼저 검증하고 나중에 개발합니다. 불필요한 개발 비용과 시간을 줄일 수 있습니다.
2. 기술보다 사용자의 문제에서 출발합니다. 좋은 AI 모델을 사용하더라도 이용 방식이 불편하면 서비스가 선택받기 어렵습니다.
3. 설계를 문서로 남깁니다. 코딩 에이전트와 협업할 때 판단 기준이 되고, 기능 추가와 운영 개선에도 활용할 수 있습니다.

ASP나 My-Story-Maker를 개발할 때도 먼저 사용자가 어떤 상황에서 서비스를 이용하는지 정하고, 핵심 기능을 작은 규모로 검증한 다음 PRD와 아키텍처를 구체화하는 접근이 적합합니다.

5. 주관식 퀴즈: 회고 답안 예시

질문: 섹션 1에서 작성한 AI 기능의 사용 경로와 내부 호출 순서를 떠올리며, 처음 답에서 가장 많이 바뀐 생각 하나와 그 근거를 적어 보세요.

아래는 강의 내용을 바탕으로 작성한 예시입니다. 실제로 처음 생각과 달라졌던 부분에 맞춰 수정하시면 됩니다.

처음에는 AI 기능을 구현하는 데 집중해 사용자가 버튼을 누르면 AI가 결과를 생성해 보여주는 정도로 생각했습니다. 하지만 오늘 PRD와 시퀀스 다이어그램을 작성하면서 **AI 기능의 구현보다 사용자가 이용하는 전체 과정과 예외 상황을 설계하는 것이 중요하다**는 점을 배웠습니다.

예를 들어 AI 응답이 늦거나 외부 API 호출이 실패했을 때의 안내, 데이터 저장 여부와 소유권, API 키의 보안까지 미리 결정해야 합니다. 또한 사용자가 실제로 어떻게 행동하는지 코드 없이 검증한 뒤 설계를 수정할 수 있다는 점도 새롭게 이해했습니다.

따라서 앞으로는 코딩 에이전트에게 바로 개발을 요청하기보다 사용 맥락과 처리 순서를 먼저 정리하고, 프로토타입으로 검증한 뒤 PRD와 아키텍처를 확정하겠습니다.

6. 반드시 기억할 내용

- 제공 채널은 기능 자체보다 사용자의 이용 맥락에 따라 결정합니다.
- 기성 AI 도구와 프로토타입으로 먼저 검증하고, 필요한 경우 개발합니다.
- 처리 시간과 오류의 영향도에 따라 동기·비동기·스케줄 실행 및 안전장치를 선택합니다.
- 시퀀스 다이어그램에서 아키텍처와 데이터 모델을 도출합니다.
- PRD에 접근 제어, 데이터, 비용, 파일럿, 아키텍처를 함께 기록합니다.
- 배포와 운영 기술은 PRD의 요구사항을 실제 서비스로 만드는 방법입니다.

한 문장 요약

좋은 AI 서비스는 모델을 연결하는 것에서 끝나지 않고, 사용 맥락을 이해하고 행동으로 검증한 뒤, 처리 흐름·시스템 구조·데이터·비용을 PRD로 정리하여 실제 운영 가능한 서비스로 구현하는 것입니다.